7 Ulysses' Compass

Mikołaj Kopernik (also known as Nicolaus Copernicus, 1473–1543): Polish astronomer, ecclesiastical lawyer, and blasphemer. Famous for his heliocentric model of the solar system, Kopernik argued for replacing the geocentric model, because the heliocentric model was more "harmonious." This position eventually lead (decades later) to Galileo's famous disharmony with, and trial by, the Church.

This story has become a fable of science's triumph over ideology and superstition. But Kopernik's justification looks poor to us now, ideology aside. There are two problems: The model was neither particularly harmonious nor more accurate than the geocentric model. The Copernican model was very complicated. In fact, it had similar epicycle clutter as the Ptolemaic model (Figure 7.1). Kopernik had moved the Sun to the center, but since he still used perfect circles for orbits, he still needed epicycles. And so "harmony" doesn't quite describe the model's appearance. Just like the Ptolemaic model, the Kopernikan model was effectively a Fourier series, a means of approximating periodic functions. This leads to the second problem: The heliocentric model made exactly the same predictions as the geocentric model. Equivalent approximations can be constructed whether the Earth is stationary or rather moving. So there was no reason to prefer it on the basis of accuracy alone.

Kopernik didn't appeal just to some vague harmony, though. He also argued for the superiority of his model on the basis of needing fewer causes: "We thus follow Nature, who producing nothing in vain or superfluous often prefers to endow one cause with many effects."[98] And it was true that a heliocentric model required fewer circles and epicycles to make the same predictions as a geocentric model. In this sense, it was *simpler*.

Scholars often prefer simpler theories. This preference is sometimes vague—a kind of aesthetic preference. Other times we retreat to pragmatism, preferring simpler theories because their simpler models are easier to work with. Frequently, scientists cite a loose principle known as **Ockham's razor**: *Models with fewer assumptions are to be preferred.* In the case of Kopernik and Ptolemy, the razor makes a clear recommendation. It cannot guarantee that Kopernik was right (he wasn't, after all), but since the heliocentric and geocentric models make the same predictions, at least the razor offers a clear resolution to the dilemma. But the razor can be hard to use more generally, because usually we must choose among models that differ in both their accuracy and their simplicity. How are we to trade these different criteria against one another? The razor offers no guidance.

This chapter describes some of the most commonly used tools for coping with this trade-off. Some notion of simplicity usually features in all of these tools, and so each is commonly compared to Ockham's razor. But each tool is equally about improving predictive accuracy. So they are not like the razor, because they explicitly trade-off accuracy and simplicity.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

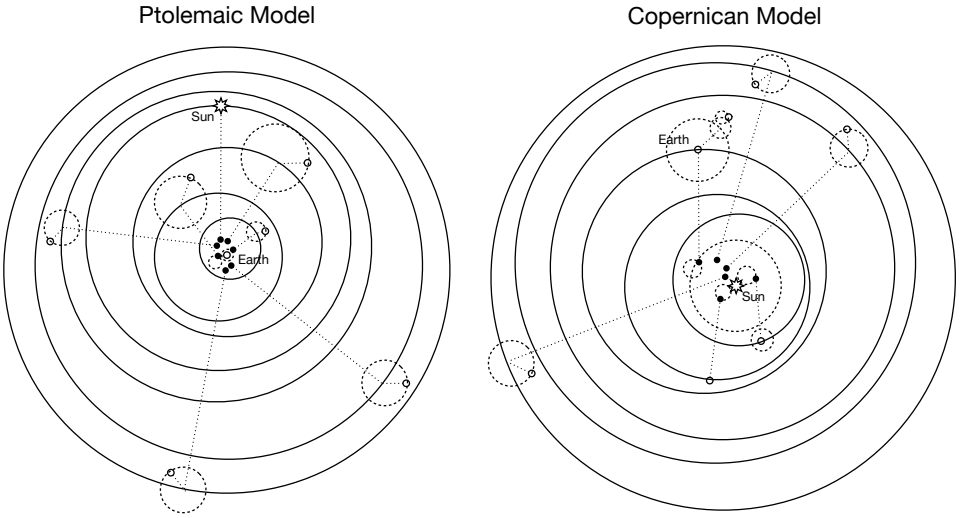


FIGURE 7.1. Ptolemaic (left) and Copernican (right) models of the solar system. Both models use epicycles (circles on circles), and both models produce exactly the same predictions. However, the Copernican model requires fewer circles. (Not all Ptolemaic epicycles are visible in the figure.)

So instead of Ockham's razor, think of Ulysses' compass. Ulysses was the hero of Homer's *Odyssey*. During his voyage, Ulysses had to navigate a narrow straight between the many-headed beast Scylla—who attacked from a cliff face and gobbled up sailors—and the sea monster Charybdis—who pulled boats and men down to a watery grave. Passing too close to either meant disaster. In the context of scientific models, you can think of these monsters as representing two fundamental kinds of statistical error:

(1) The many-headed beast of **OVERFITTING**, which leads to poor prediction by learning too much from the data
(2) The whirlpool of **UNDERFITTING**, which leads to poor prediction by learning too little from the data

There is a third monster, the one you met in previous chapters—confounding. In this chapter you'll see that confounded models can in fact produce better predictions than models that correctly measure a causal relationship. The consequence of this is that, when we design any particular statistical model, we must decide whether we want to understand causes or rather just predict. These are not the same goal, and different models are needed for each. However, to accurately measure a causal influence, we still have to deal with overfitting. The monsters of overfitting and underfitting are always lurking, no matter the goal.

Our job is to carefully navigate among these monsters. There are two common families of approaches. The first approach is to use a **REGULARIZING PRIOR** to tell the model not to get too excited by the data. This is the same device that non-Bayesian methods refer to as "penalized likelihood." The second approach is to use some scoring device, like **INFORMATION CRITERIA** or **CROSS-VALIDATION**, to model the prediction task and estimate predictive accuracy. Both families of approaches are routinely used in the natural and social sciences. Furthermore, they can be—maybe should be—used in combination. So it's worth understanding both, as you're going to need both at some point.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

In order to introduce information criteria, this chapter must also introduce **INFORMATION THEORY**. If this is your first encounter with information theory, it'll probably seem strange. But some understanding of it is needed. Once you start using information criteria—this chapter describes AIC, DIC, WAIC, and PSIS—you'll find that implementing them is much easier than understanding them. This is their curse. So most of this chapter aims to fight the curse, focusing on their conceptual foundations, with applications to follow.

It's worth noting, before getting started, that this material is hard. If you find yourself confused at any point, you are normal. Any sense of confusion you feel is just your brain correctly calibrating to the subject matter. Over time, confusion is replaced by comprehension for how overfitting, regularization, and information criteria behave in familiar contexts.

Rethinking: Stargazing. The most common form of model selection among practicing scientists is to search for a model in which every coefficient is statistically significant. Statisticians sometimes call this **STARGAZING**, as it is embodied by scanning for asterisks (**) trailing after estimates. A colleague of mine once called this approach the "Space Odyssey," in honor of A. C. Clarke's novel and film. The model that is full of stars, the thinking goes, is best.

But such a model is not best. Whatever you think about null hypothesis significance testing in general, using it to select among structurally different models is a mistake—*p*-values are not designed to help you navigate between underfitting and overfitting. As you'll see once you start using AIC and related measures, predictor variables that improve prediction are not always statistically significant. It is also possible for variables that are statistically significant to do nothing useful for prediction. Since the conventional 5% threshold is purely conventional, we shouldn't expect it to optimize anything.

Rethinking: Is AIC Bayesian? AIC is not usually thought of as a Bayesian tool. There are both historical and statistical reasons for this. Historically, AIC was originally derived without reference to Bayesian probability. Statistically, AIC uses MAP estimates instead of the entire posterior, and it requires flat priors. So it doesn't look particularly Bayesian. Reinforcing this impression is the existence of another model comparison metric, the **BAYESIAN INFORMATION CRITERION** (BIC). However, BIC also requires flat priors and MAP estimates, although it's not actually an "information criterion."

Regardless, AIC has a clear and pragmatic interpretation under Bayesian probability, and Akaike and others have long argued for alternative Bayesian justifications of the procedure.[99] And as you'll see later in the book, more obviously Bayesian information criteria like WAIC provide almost exactly the same results as AIC, when AIC's assumptions are met. In this light, we can fairly regard AIC as a special limit of a Bayesian criterion like WAIC, even if that isn't how AIC was originally derived. All of this is an example of a common feature of statistical procedures: The same procedure can be derived and justified from multiple, sometimes philosophically incompatible, perspectives.

7.1. The problem with parameters

In the previous chapters, we saw how adding variables and parameters to a model can help to reveal hidden effects and improve estimates. You also saw that adding variables can hurt, in particular when we lack a trusted causal model. Colliders are real. But sometimes we don't care about causal inference. Maybe we just want to make good predictions. Consider for example the grandparent-parent-child example from the previous chapter. Just adding all the variables to the model will give us a good predictive model in that case. That we don't understand what is going on is irrelevant. So is just adding everything to the model okay?

The answer is "no." There are two related problems with just adding variables. The first is that adding parameters—making the model more complex—nearly always improves the

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

fit of a model to the data.[100] By "fit" I mean a measure of how well the model can retrodict the data used to fit the model. There are many such measures, each with its own foibles. In the context of linear Gaussian models, R^2 is the most common measure of this kind. Often described as "variance explained," R^2 is defined as:

$$R^2 = \frac{\text{var(outcome)} - \text{var(residuals)}}{\text{var(outcome)}} = 1 - \frac{\text{var(residuals)}}{\text{var(outcome)}}$$

Being easy to compute, R^2 is popular. Like other measures of fit to sample, R^2 increases as more predictor variables are added. This is true even when the variables you add to a model are just random numbers, with no relation to the outcome. So it's no good to choose among models using only fit to the data.

Second, while more complex models fit the data better, they often predict new data worse. Models that have many parameters tend to *overfit* more than simpler models. This means that a complex model will be very sensitive to the exact sample used to fit it, leading to potentially large mistakes when future data is not exactly like the past data. But simple models, with too few parameters, tend instead to *underfit*, systematically over-predicting or under-predicting the data, regardless of how well future data resemble past data. So we can't always favor either simple models or complex models.

Let's examine both of these issues in the context of a simple example.

7.1.1. More parameters (almost) always improve fit. OVERFITTING occurs when a model learns too much from the sample. What this means is that there are both *regular* and *irregular* features in every sample. The regular features are the targets of our learning, because they generalize well or answer a question of interest. Regular features are useful, given an objective of our choice. The irregular features are instead aspects of the data that do not generalize and so may mislead us.

Overfitting happens automatically, unfortunately. In the kind of statistical models we've seen so far in this book, adding additional parameters will always improve the fit of a model to the sample. Later in the book, beginning with Chapter 13, you'll meet models for which adding parameters does not necessarily improve fit to the sample, but may well improve predictive accuracy.

Here's an example of overfitting. The data displayed in FIGURE 7.2 are average brain volumes and body masses for seven hominin species.[101] Let's get these data into R, so you can work with them. I'm going to build these data from direct input, rather than loading a pre-made data frame, just so you see an example of how to build a data frame from scratch.

R code 7.1

```
sppnames <- c( "afarensis","africanus","habilis","boisei",
    "rudolfensis","ergaster","sapiens")
brainvolcc <- c( 438 , 452 , 612, 521, 752, 871, 1350 )
masskg <- c( 37.0 , 35.5 , 34.5 , 41.5 , 55.5 , 61.0 , 53.5 )
d <- data.frame( species=sppnames , brain=brainvolcc , mass=masskg )
```

Now you have a data frame, `d`, containing the brain size and body size values. It's not unusual for data like this to be highly correlated—brain size is correlated with body size, across species. A standing question, however, is to what extent particular species have brains that are larger than we'd expect, after taking body size into account. A common solution is to fit a linear regression that models brain size as a linear function of body size. Then the remaining

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

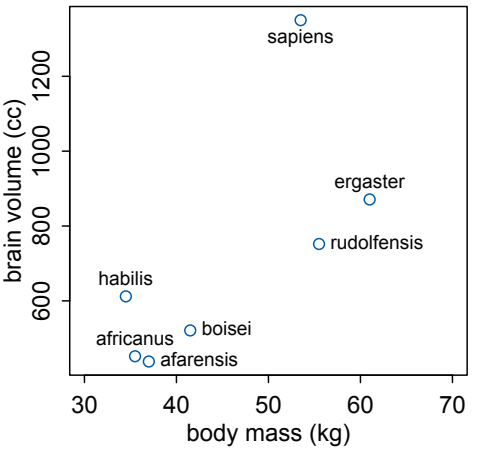


FIGURE 7.2. Average brain volume in cubic centimeters against body mass in kilograms, for six hominin species. What model best describes the relationship between brain size and body size?

variation in brain size can be modeled as a function of other variables, like ecology or diet. This is the same "statistical control" strategy explained in previous chapters.

Controlling for body size, however, depends upon having a good functional mapping of the association between body size and brain size. We've just used linear functions so far. But why use a line to relate body size to brain size? It's not clear why nature demands that the relationship among species be a straight line. Why not consider a curved model, like a parabola? Indeed, why not a cubic function of body size, or even a spline? There's no reason to suppose *a priori* that brain size scales only linearly with body size. Indeed, many readers will prefer to model a linear relationship between log brain volume and log body mass (an exponential relationship). But that's not the direction I'm headed with this example. The lesson here will arise, no matter how we transform the data.

Let's fit a series of increasingly complex model families and see which function fits the data best. We'll use polynomial regressions, so review Section 4.5 (page 110) if necessary. Importantly, recall that polynomial regressions are common, but usually a bad idea. In this example, I will show you that they can be a very bad idea when used blindly. But the splines from Chapter 4 will suffer the same basic problem. In the practice problems at the end of the chapter, you will return to this example and try it with splines.

The simplest model that relates brain size to body size is the linear one. It will be the first model we consider. Before writing out the model, let's rescale the variables. Recall from earlier chapters that rescaling predictor and outcome variables is often helpful in getting the model to fit and in specifying and understanding the priors. In this case, we want to standardize body mass—give it mean zero and standard deviation one—and rescale the outcome, brain volume, so that the largest observed value is 1. Why not standardize brain volume as well? Because we want to preserve zero as a reference point: No brain at all. You can't have negative brain. I don't think.

R code 7.2

```
d$mass_std <- (d$mass - mean(d$mass))/sd(d$mass)
d$brain_std <- d$brain / max(d$brain)
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Now here's the mathematical version of the first linear model. The only trick to note is the log-normal prior on σ. This will make it easier to keep σ positive, as it should be.

$$\begin{aligned} b_i &\sim \text{Normal}(\mu_i, \sigma) \\ \mu_i &= \alpha + \beta m_i \\ \alpha &\sim \text{Normal}(0.5, 1) \\ \beta &\sim \text{Normal}(0, 10) \\ \sigma &\sim \text{Log-Normal}(0, 1) \end{aligned}$$

This simply says that the average brain volume b_i of species i is a linear function of its body mass m_i. Now consider what the priors imply. The prior for α is just centered on the mean brain volume (rescaled) in the data. So it says that the average species with an average body mass has a brain volume with an 89% credible interval from about -1 to 2. That is ridiculously wide and includes impossible (negative) values. The prior for β is very flat and centered on zero. It allows for absurdly large positive and negative relationships. These priors allow for absurd inferences, especially as the model gets more complex. And that's part of the lesson, so let's continue to fit the model now:

R code 7.3

```
m7.1 <- quap(
    alist(
        brain_std ~ dnorm( mu , exp(log_sigma) ),
        mu <- a + b*mass_std,
        a ~ dnorm( 0.5 , 1 ),
        b ~ dnorm( 0 , 10 ),
        log_sigma ~ dnorm( 0 , 1 )
    ), data=d )
```

I've used `exp(log_sigma)` in the likelihood, so that the result is always greater than zero.

Rethinking: OLS and Bayesian anti-essentialism. It would be possible to use **ORDINARY LEAST SQUARES** (OLS) to get posterior distributions for these brain size models. For example, you could use R's simple `lm` function to get the posterior distribution for `m6.1`. You won't get a posterior for `sigma` however.

R code 7.4

```
m7.1_OLS <- lm( brain_std ~ mass_std , data=d )
post <- extract.samples( m7.1_OLS )
```

OLS is not considered a Bayesian algorithm. But as long as the priors are vague, minimizing the sum of squared deviations to the regression line is equivalent to finding the posterior mean. In fact, Carl Friedrich Gauss originally derived the OLS procedure in a Bayesian framework.[102] Back then, nearly all probability was Bayesian, although the term "Bayesian" wouldn't be used much until the twentieth century. In most cases, a non-Bayesian procedure will have an approximate Bayesian interpretation. This fact is powerful in both directions. The Bayesian interpretation of a non-Bayesian procedure recasts assumptions in terms of information, and this can be very useful for understanding why a procedure works. Likewise, a Bayesian model can be embodied in an efficient, but approximate, "non-Bayesian" procedure. Bayesian inference means approximating the posterior distribution. It does not specify how that approximation is done.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Before pausing to plot the posterior distribution, like we did in previous chapters, let's focus on the R^2, the proportion of variance "explained" by the model. What is really meant here is that the linear model retrodicts some proportion of the total variation in the outcome data it was fit to. The remaining variation is just the variation of the residuals (page 135).

The point of this example is not to praise R^2 but to bury it. But we still need to compute it before burial. This is thankfully easy. We just compute the posterior predictive distribution for each observation—you did this in earlier chapters with `sim`. Then we subtract each observation from its prediction to get a residual. Then we need the variance of both these residuals and the outcome variable. This means the *actual* empirical variance, not the variance that R returns with the `var` function, which is a frequentist estimator and therefore has the wrong denominator. So we'll compute variance the old fashioned way: the average squared deviation from the mean. The `rethinking` package includes a function `var2` for this purpose. In principle, the Bayesian approach mandates that we do this for each sample from the posterior. But R^2 is traditionally computed only at the mean prediction. So we'll do that as well here. Later in the chapter you'll learn a properly Bayesian score that uses the entire posterior distribution.

R code 7.5

```
set.seed(12)
s <- sim( m7.1 )
r <- apply(s,2,mean) - d$brain_std
resid_var <- var2(r)
outcome_var <- var2( d$brain_std )
1 - resid_var/outcome_var
```

```
[1] 0.4774589
```

We'll want to do this for the next several models, so let's write a function to make it repeatable. If you find yourself writing code more than once, it is usually saner to write a function and call the function more than once instead.

R code 7.6

```
R2_is_bad <- function( quap_fit ) {
    s <- sim( quap_fit , refresh=0 )
    r <- apply(s,2,mean) - d$brain_std
    1 - var2(r)/var2(d$brain_std)
}
```

Now for some other models to compare to `m7.1`. We'll consider five more models, each more complex than the last. Each of these models will just be a polynomial of higher degree. For example, a second-degree polynomial that relates body size to brain size is a parabola. In math form, it is:

$$
\begin{aligned}
b_i &\sim \text{Normal}(\mu_i, \sigma) \\
\mu_i &= \alpha + \beta_1 m_i + \beta_2 m_i^2 \\
\alpha &\sim \text{Normal}(0.5, 1) \\
\beta_j &\sim \text{Normal}(0, 10) \qquad \text{for } j = 1..2 \\
\sigma &\sim \text{Log-Normal}(0, 1)
\end{aligned}
$$

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

This model family adds one more parameter, β_2, but uses all of the same data as `m7.1`. To do this model in `quap`, we can define β as a vector. The only trick required is to tell `quap` how long that vector is by using a `start` list:

R code 7.7

```
m7.2 <- quap(
    alist(
        brain_std ~ dnorm( mu , exp(log_sigma) ),
        mu <- a + b[1]*mass_std + b[2]*mass_std^2,
        a ~ dnorm( 0.5 , 1 ),
        b ~ dnorm( 0 , 10 ),
        log_sigma ~ dnorm( 0 , 1 )
    ), data=d , start=list(b=rep(0,2)) )
```

The next four models are constructed in similar fashion. The models `m7.3` through `m7.6` are just third-degree, fourth-degree, fifth-degree, and sixth-degree polynomials.

R code 7.8

```
m7.3 <- quap(
    alist(
        brain_std ~ dnorm( mu , exp(log_sigma) ),
        mu <- a + b[1]*mass_std + b[2]*mass_std^2 +
                  b[3]*mass_std^3,
        a ~ dnorm( 0.5 , 1 ),
        b ~ dnorm( 0 , 10 ),
        log_sigma ~ dnorm( 0 , 1 )
    ), data=d , start=list(b=rep(0,3)) )

m7.4 <- quap(
    alist(
        brain_std ~ dnorm( mu , exp(log_sigma) ),
        mu <- a + b[1]*mass_std + b[2]*mass_std^2 +
                  b[3]*mass_std^3 + b[4]*mass_std^4,
        a ~ dnorm( 0.5 , 1 ),
        b ~ dnorm( 0 , 10 ),
        log_sigma ~ dnorm( 0 , 1 )
    ), data=d , start=list(b=rep(0,4)) )

m7.5 <- quap(
    alist(
        brain_std ~ dnorm( mu , exp(log_sigma) ),
        mu <- a + b[1]*mass_std + b[2]*mass_std^2 +
                  b[3]*mass_std^3 + b[4]*mass_std^4 +
                  b[5]*mass_std^5,
        a ~ dnorm( 0.5 , 1 ),
        b ~ dnorm( 0 , 10 ),
        log_sigma ~ dnorm( 0 , 1 )
    ), data=d , start=list(b=rep(0,5)) )
```

That last model, `m7.6`, has one trick in it. The standard deviation is replaced with a constant value 0.001. The model will not work otherwise, for a very important reason that will become clear as we plot these monsters. Here's the last model:

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

R code 7.9

```
m7.6 <- quap(
    alist(
        brain_std ~ dnorm( mu , 0.001 ),
        mu <- a + b[1]*mass_std + b[2]*mass_std^2 +
                  b[3]*mass_std^3 + b[4]*mass_std^4 +
                  b[5]*mass_std^5 + b[6]*mass_std^6,
        a ~ dnorm( 0.5 , 1 ),
        b ~ dnorm( 0 , 10 )
    ), data=d , start=list(b=rep(0,6)) )
```

Now to plot each model. We'll follow the steps from earlier chapters: extract samples from the posterior, compute the posterior predictive distribution at each of several locations on the horizontal axis, summarize, and plot. For `m7.1`:

R code 7.10

```
post <- extract.samples(m7.1)
mass_seq <- seq( from=min(d$mass_std) , to=max(d$mass_std) , length.out=100 )
l <- link( m7.1 , data=list( mass_std=mass_seq ) )
mu <- apply( l , 2 , mean )
ci <- apply( l , 2 , PI )
plot( brain_std ~ mass_std , data=d )
lines( mass_seq , mu )
shade( ci , mass_seq )
```

I show this plot and all the others, with some cosmetic improvements (see `brain_plot` for the code), in FIGURE 7.3. Each plot also displays R^2. As the degree of the polynomial defining the mean increases, the R^2 always improves, indicating better retrodiction of the data. The fifth-degree polynomial has an R^2 value of 0.99. It almost passes exactly through each point. The sixth-degree polynomial actually does pass through every point, and it has no residual variance. It's a perfect fit, $R^2 = 1$. That is why we had to fix the `sigma` value—if it were estimated, it would shrink to zero, because the residual variance is zero when the line passes right through the center of each point.

However, you can see from looking at the paths of the predicted means that the higher-degree polynomials are increasingly absurd. This absurdity is seen most easily in FIGURE 7.3, `m7.6`, the most complex model. The fit is perfect, but the model is ridiculous. Notice that there is a gap in the body mass data, because there are no fossil hominins with body mass between 55 kg and about 60 kg. In this region, the predicted mean brain size from the high-degree polynomial models has nothing to predict, and so the models pay no price for swinging around wildly in this interval. The swing is so extreme that I had to extend the range of the vertical axis to display the depth at which the predicted mean finally turns back around. At around 58 kg, the model predicts a negative brain size! The model pays no price (yet) for this absurdity, because there are no cases in the data with body mass near 58 kg.

Why does the sixth-degree polynomial fit perfectly? Because it has enough parameters to assign one to each point of data. The model's equation for the mean has 7 parameters:

$$\mu_i = \alpha + \beta_1 m_i + \beta_2 m_i^2 + \beta_3 m_i^3 + \beta_4 m_i^4 + \beta_5 m_i^5 + \beta_6 m_i^6$$

and there are 7 species to predict brain sizes for. So effectively, this model assigns a unique parameter to reiterate each observed brain size. This is a general phenomenon: If you adopt

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

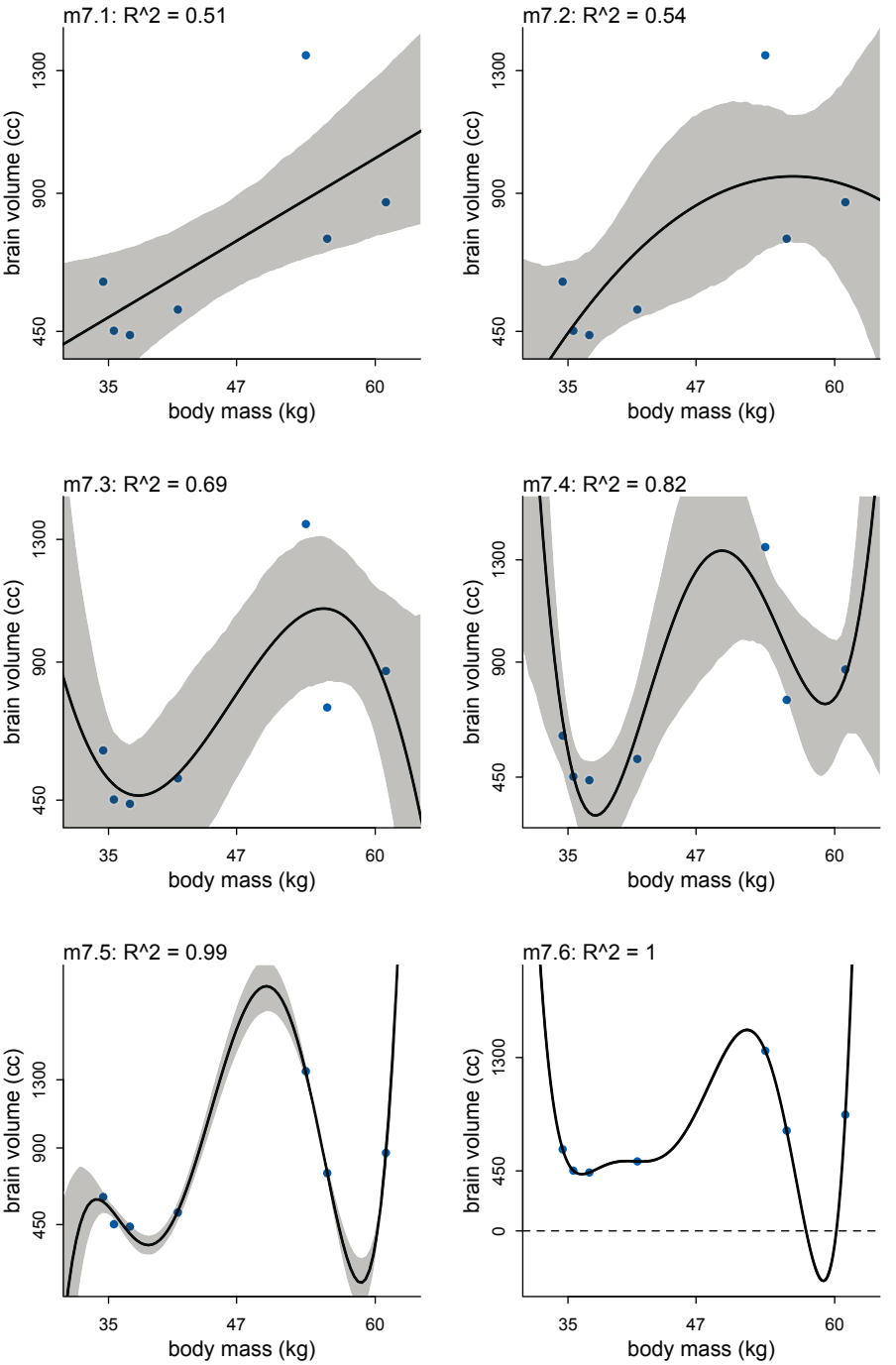


FIGURE 7.3. Polynomial linear models of increasing degree for the hominin data. Each plot shows the posterior mean in black, with 89% interval of the mean shaded. R^2 is displayed above each plot. In order from top-left: First-degree polynomial, second-degree, third-degree, fourth-degree, fifth-degree, and sixth-degree.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

a model family with enough parameters, you can fit the data exactly. But such a model will make rather absurd predictions for yet-to-be-observed cases.

Rethinking: Model fitting as compression. Another perspective on the absurd model just above is to consider that model fitting can be considered a form of DATA COMPRESSION. Parameters summarize relationships among the data. These summaries compress the data into a simpler form, although with loss of information ("lossy" compression) about the sample. The parameters can then be used to generate new data, effectively decompressing the data.

When a model has a parameter to correspond to each datum, such as `m7.6`, then there is actually no compression. The model just encodes the raw data in a different form, using parameters instead. As a result, we learn nothing about the data from such a model. Learning about the data requires using a simpler model that achieves some compression, but not too much. This view of model selection is often known as MINIMUM DESCRIPTION LENGTH (MDL).[103]

7.1.2. Too few parameters hurts, too.

The overfit polynomial models fit the data extremely well, but they suffer for this within-sample accuracy by making nonsensical out-of-sample predictions. In contrast, UNDERFITTING produces models that are inaccurate both within and out of sample. They learn too little, failing to recover regular features of the sample.

Another way to conceptualize an underfit model is to notice that it is insensitive to the sample. We could remove any one point from the sample and get almost the same regression line. In contrast, the most complex model, `m7.6`, is very sensitive to the sample. If we removed any one point, the mean would change a lot. You can see this sensitivity in FIGURE 7.4. In both plots what I've done is drop each row of the data, one at a time, and re-derive the posterior distribution. On the left, each line is a first-degree polynomial, `m7.1`, fit to one of the seven possible sets of data constructed from dropping one row. The curves on the right are instead different fourth-order polynomials, `m7.4`. Notice that the straight lines hardly vary, while the curves fly about wildly. This is a general contrast between underfit and overfit models: sensitivity to the exact composition of the sample used to fit the model.

Overthinking: Dropping rows. The calculations needed to produce FIGURE 7.4 are made easy by a trick of R's index notation. To drop a row `i` from a data frame `d`, just use:

```
d_minus_i <- d[ -i , ]
```

R code 7.11

This means *drop the i-th row and keep all of the columns.* Repeating the regression is then just a matter of looping over the rows. Look inside the function `brain_loo_plot` in the `rethinking` package to see how the figure was drawn and explore other models.

Rethinking: Bias and variance. The underfitting/overfitting dichotomy is often described as the BIAS-VARIANCE TRADE-OFF.[104] While not exactly the same distinction, the bias-variance trade-off addresses the same problem. "Bias" is related to underfitting, while "variance" is related to overfitting. These terms are confusing, because they are used in many different ways in different contexts, even within statistics. The term "bias" also sounds like a bad thing, even though increasing bias often leads to better predictions.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

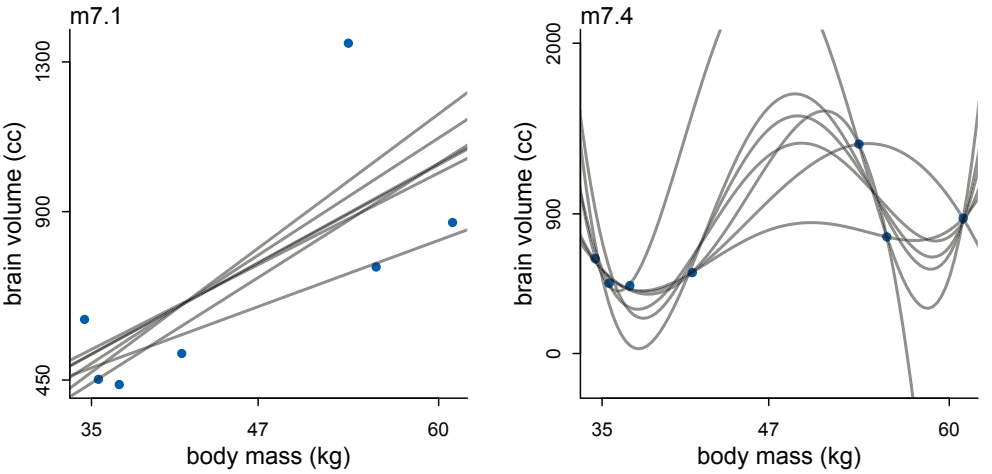


FIGURE 7.4. Underfitting and overfitting as under-sensitivity and over-sensitivity to sample. In both plots, a regression is fit to the seven sets of data made by dropping one row from the original data. Left: An underfit model is insensitive to the sample, changing little as individual points are dropped. Right: An overfit model is sensitive to the sample, changing dramatically as points are dropped.

7.2. Entropy and accuracy

So how do we navigate between the hydra of overfitting and the vortex of underfitting? Whether you end up using regularization or information criteria or both, the first thing you must do is pick a criterion of model performance. What do you want the model to do well at? We'll call this criterion the *target*, and in this section you'll see how information theory provides a common and useful target.

The path to out-of-sample deviance is twisty, however. Here are the steps ahead. First, we need to establish a measurement scale for distance from perfect accuracy. This will require a little *information theory*, as it will provide a natural measurement scale for the distance between two probability distributions. Second, we need to establish *deviance* as an approximation of relative distance from perfect accuracy. Finally, we must establish that it is only deviance out-of-sample that is of interest. Once you have deviance in hand as a measure of model performance, in the sections to follow you'll see how both regularizing priors and information criteria help you improve and estimate the out-of-sample deviance of a model.

This material is complicated. You don't have to understand everything on the first pass.

7.2.1. Firing the weatherperson. Accuracy depends upon the definition of the target, and there is no universally best target. In defining a target, there are two major dimensions to worry about:

(1) *Cost-benefit analysis.* How much does it cost when we're wrong? How much do we win when we're right? Most scientists never ask these questions in any formal way, but applied scientists must routinely answer them.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

(2) *Accuracy in context.* Some prediction tasks are inherently easier than others. So even if we ignore costs and benefits, we still need a way to judge "accuracy" that accounts for how much a model could possibly improve prediction.

It will help to explore these two dimensions in an example. Suppose in a certain city, a certain weatherperson issues uncertain predictions for rain or shine on each day of the year.[105] The predictions are in the form of probabilities of rain. The currently employed weatherperson predicted these chances of rain over a 10-day sequence, with the actual outcomes shown below each prediction:

Day	1	2	3	4	5	6	7	8	9	10
Prediction	1	1	1	0.6	0.6	0.6	0.6	0.6	0.6	0.6
Observed										

A newcomer rolls into town and boasts that he can best the current weatherperson by always predicting sunshine. Over the same 10-day period, the newcomer's record would be:

Day	1	2	3	4	5	6	7	8	9	10
Prediction	0	0	0	0	0	0	0	0	0	0
Observed										

"So by rate of correct prediction alone," the newcomer announces, "I'm the best person for the job."

The newcomer is right. Define *hit rate* as the average chance of a correct prediction. So for the current weatherperson, she gets $3 \times 1 + 7 \times 0.4 = 5.8$ hits in 10 days, for a rate of $5.8/10 = 0.58$ correct predictions per day. In contrast, the newcomer gets $3 \times 0 + 7 \times 1 = 7$, for $7/10 = 0.7$ hits per day. The newcomer wins.

7.2.1.1. *Costs and benefits.* But it's not hard to find another criterion, other than rate of correct prediction, that makes the newcomer look foolish. Any consideration of costs and benefits will suffice. Suppose for example that you hate getting caught in the rain, but you also hate carrying an umbrella. Let's define the cost of getting wet as -5 points of happiness and the cost of carrying an umbrella as -1 point of happiness. Suppose your chance of carrying an umbrella is equal to the forecast probability of rain. Your job is now to maximize your happiness by choosing a weatherperson. Here are your points, following either the current weatherperson or the newcomer:

Day	1	2	3	4	5	6	7	8	9	10
Observed										
Points										
Current	-1	-1	-1	-0.6	-0.6	-0.6	-0.6	-0.6	-0.6	-0.6
Newcomer	-5	-5	-5	0	0	0	0	0	0	0

So the current weatherperson nets you $3 \times (-1) + 7 \times (-0.6) = -7.2$ happiness, while the newcomer nets you -15 happiness. So the newcomer doesn't look so clever now. You can play around with the costs and the decision rule, but since the newcomer always gets you caught unprepared in the rain, it's not hard to beat his forecast.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

7.2.1.2. *Measuring accuracy.* But even if we ignore costs and benefits of any actual decision based upon the forecasts, there's still ambiguity about which measure of "accuracy" to adopt. There's nothing special about "hit rate." The question to focus on is: Which definition of "accuracy" is maximized by knowing the true model generating the data? Surely we can't do better than that.

Consider computing the probability of predicting the exact sequence of days. This means computing the probability of a correct prediction for each day. Then multiply all of these probabilities together to get the joint probability of correctly predicting the observed sequence. This is the same thing as the joint likelihood, which you've been using up to this point to fit models with Bayes' theorem. This is the definition of accuracy that is maximized by the correct model.

In this light, the newcomer looks even worse. The probability for the current weatherperson is $1^3 \times 0.4^7 \approx 0.005$. For the newcomer, it's $0^3 \times 1^7 = 0$. So the newcomer has zero probability of getting the sequence correct. This is because the newcomer's predictions never expect rain. So even though the newcomer has a high *average* probability of being correct (hit rate), he has a terrible *joint* probability of being correct.

And the joint probability is the measure we want. Why? Because it appears in Bayes' theorem as the likelihood. It's the unique measure that correctly counts up the relative number of ways each event (sequence of rain and shine) could happen. Another way to think of this is to consider what happens when we maximize average probability or joint probability. The true data-generating model will not have the highest hit rate. You saw this already with the weatherperson: Assigning zero probability to rain improves hit rate, but it is clearly wrong. In contrast, the true model will have the highest joint probability.

In the statistics literature, you will sometimes see this measure of accuracy called the **LOG SCORING RULE**, because typically we compute the logarithm of the joint probability and report that. If you see an analysis using something else, either it is a special case of the log scoring rule or it is possibly much worse.

Rethinking: Calibration is overrated. It's common for models to be judged by their **CALIBRATION**. If a model predicts a 40% chance of rain, then it is said to be "calibrated" if it actually rains on 40% of such predictions. The problem is that calibrated predictions do not have to be good. For example, if it rains on 40% of days, then a model that just predicts a 40% chance of rain on every day will be perfectly calibrated. But it will also be terribly inaccurate. Nor do good predictions have to be calibrated. Suppose a forecaster always has 100% confidence in each forecast and correctly predicts the weather on 80% of days. The forecaster is accurate, but he is not calibrated. He is overconfident.

Here's a real example. The forecasting website www.fivethirtyeight.com makes many predictions. Their calibration for sporting events is almost perfect.[106] But their accuracy is often barely better than guessing. In contrast, their political predictions are less calibrated, but more accurate on average.

Terms like "calibration" have various meanings. So it's good to provide and ask for contextual definitions.[107] The posterior predictive checks endorsed in this book, for example, are sometimes called "calibration checks."

7.2.2. Information and uncertainty. So we want to use the log probability of the data to score the accuracy of competing models. The next problem is how to measure distance from perfect prediction. A perfect prediction would just report the true probabilities of rain on each day. So when either weatherperson provides a prediction that differs from the target, we can measure the distance of the prediction from the target. But what kind of distance

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

should we adopt? It's not obvious how to go about answering this question. But there turns out to be a unique and optimal answer.

Getting to the answer depends upon appreciating what an accuracy metric needs to do. It should appreciate that some targets are just easier to hit than other targets. For example, suppose we extend the weather forecast into the winter. Now there are three types of days: rain, sun, and snow. Now there are three ways to be wrong, instead of just two. This has to be reflected in any reasonable measure of distance from the target, because by adding another type of event, the target has gotten harder to hit.

It's like taking a two-dimensional archery bullseye and forcing the archer to hit the target at the right *time*—a third dimension—as well. Now the possible distance between the best archer and the worst archer has grown, because there's another way to miss. And with another way to miss, one might also say that there is another way for an archer to impress. As the potential distance between the target and the shot increases, so too does the potential improvement and ability of a talented archer to impress us.

The solution to the problem of how to measure distance of a model's accuracy from a target was provided in the late 1940s.[108] Originally applied to problems in communication of messages, such as telegraph, the field of **INFORMATION THEORY** is now important across the basic and applied sciences, and it has deep connections to Bayesian inference. And like many successful fields, information theory has spawned many bogus applications, as well.[109]

The basic insight is to ask: *How much is our uncertainty reduced by learning an outcome?* Consider the weather forecasts again. Forecasts are issued in advance and the weather is uncertain. When the actual day arrives, the weather is no longer uncertain. The reduction in uncertainty is then a natural measure of how much we have learned, how much "information" we derive from observing the outcome. So if we can develop a precise definition of "uncertainty," we can provide a baseline measure of how hard it is to predict, as well as how much improvement is possible. The measured decrease in uncertainty is the definition of *information* in this context.

> *Information*: The reduction in uncertainty when we learn an outcome.

To use this definition, what we need is a principled way to quantify the uncertainty inherent in a probability distribution. So suppose again that there are two possible weather events on any particular day: Either it is sunny or it is rainy. Each of these events occurs with some probability, and these probabilities add up to one. What we want is a function that uses the probabilities of shine and rain and produces a measure of uncertainty.

There are many possible ways to measure uncertainty. The most common way begins by naming some properties a measure of uncertainty should possess. These are the three intuitive desiderata:

(1) The measure of uncertainty should be continuous. If it were not, then an arbitrarily small change in any of the probabilities, for example the probability of rain, would result in a massive change in uncertainty.

(2) The measure of uncertainty should increase as the number of possible events increases. For example, suppose there are two cities that need weather forecasts. In the first city, it rains on half of the days in the year and is sunny on the others. In the second, it rains, shines, and hails, each on 1 out of every 3 days in the year. We'd like our measure of uncertainty to be larger in the second city, where there is one more kind of event to predict.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

(3) The measure of uncertainty should be additive. What this means is that if we first measure the uncertainty about rain or shine (2 possible events) and then the uncertainty about hot or cold (2 different possible events), the uncertainty over the four combinations of these events—rain/hot, rain/cold, shine/hot, shine/cold—should be the sum of the separate uncertainties.

There is only one function that satisfies these desiderata. This function is usually known as **INFORMATION ENTROPY**, and has a surprisingly simple definition. If there are n different possible events and each event i has probability p_i, and we call the list of probabilities p, then the unique measure of uncertainty we seek is:

$$H(p) = -\operatorname{E}\log(p_i) = -\sum_{i=1}^{n} p_i \log(p_i) \tag{7.1}$$

In plainer words:

> *The uncertainty contained in a probability distribution is the average log-probability of an event.*

"Event" here might refer to a type of weather, like rain or shine, or a particular species of bird or even a particular nucleotide in a DNA sequence.

While it's not worth going into the details of the derivation of H, it is worth pointing out that nothing about this function is arbitrary. Every part of it derives from the three requirements above. Still, we accept $H(p)$ as a useful measure of uncertainty not because of the premises that lead to it, but rather because it has turned out to be so useful and productive.

An example will help to demystify the function $H(p)$. To compute the information entropy for the weather, suppose the true probabilities of rain and shine are $p_1 = 0.3$ and $p_2 = 0.7$, respectively. Then:

$$H(p) = -\big(p_1 \log(p_1) + p_2 \log(p_2)\big) \approx 0.61$$

As an R calculation:

R code 7.12

```
p <- c( 0.3 , 0.7 )
-sum( p*log(p) )
```

```
[1] 0.6108643
```

Suppose instead we live in Abu Dhabi. Then the probabilities of rain and shine might be more like $p_1 = 0.01$ and $p_2 = 0.99$. Now the entropy would be approximately 0.06. Why has the uncertainty decreased? Because in Abu Dhabi it hardly ever rains. Therefore there's much less uncertainty about any given day, compared to a place in which it rains 30% of the time. It's in this way that information entropy measures the uncertainty inherent in a distribution of events. Similarly, if we add another kind of event to the distribution—forecasting into winter, so also predicting snow—entropy tends to increase, due to the added dimensionality of the prediction problem. For example, suppose probabilities of sun, rain, and snow are $p_1 = 0.7$, $p_2 = 0.15$, and $p_3 = 0.15$, respectively. Then entropy is about 0.82.

These entropy values by themselves don't mean much to us, though. Instead we can use them to build a measure of accuracy. That comes next.

Overthinking: More on entropy. Above I said that information entropy is the average log-probability. But there's also a -1 in the definition. Multiplying the average log-probability by -1 just makes the entropy H increase from zero, rather than decrease from zero. It's conventional, but not functional.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

The logarithms above are natural logs (base e), but changing the base rescales without any effect on inference. Binary logarithms, base 2, are just as common. As long as all of the entropies you compare use the same base, you'll be fine.

The only trick in computing H is to deal with the inevitable question of what to do when $p_i = 0$. The $\log(0) = -\infty$, which won't do. However, L'Hôpital's rule tells us that $\lim_{p_i \to 0} p_i \log(p_i) = 0$. So just assume that $0 \log(0) = 0$, when you compute H. In other words, events that never happen drop out. Just remember that when an event never happens, there's no point in keeping it in the model.

Rethinking: The benefits of maximizing uncertainty. Information theory has many applications. A particularly important application is MAXIMUM ENTROPY, also known as MAXENT. Maximum entropy is a family of techniques for finding probability distributions that are most consistent with states of knowledge. In other words, given what we know, what is the *least surprising* distribution? It turns out that one answer to this question maximizes the information entropy, using the prior knowledge as constraint.[110] If you do this, you actually end up with the posterior distribution. So Bayesian updating is entropy maximization. Maximum entropy features prominently in Chapter 10, where it will help us build generalized linear models (GLMs).

7.2.3. From entropy to accuracy. It's nice to have a way to quantify uncertainty. H provides this. So we can now say, in a precise way, how hard it is to hit the target. But how can we use information entropy to say how far a model is from the target? The key lies in DIVERGENCE:

> **Divergence:** The additional uncertainty induced by using probabilities from one distribution to describe another distribution.

This is often known as *Kullback-Leibler divergence* or simply KL divergence, named after the people who introduced it for this purpose.[111]

Suppose for example that the true distribution of events is $p_1 = 0.3, p_2 = 0.7$. If we believe instead that these events happen with probabilities $q_1 = 0.25, q_2 = 0.75$, how much additional uncertainty have we introduced, as a consequence of using $q = \{q_1, q_2\}$ to approximate $p = \{p_1, p_2\}$? The formal answer to this question is based upon H, and has a similarly simple formula:

$$D_{\text{KL}}(p, q) = \sum_i p_i \big(\log(p_i) - \log(q_i) \big) = \sum_i p_i \log \left(\frac{p_i}{q_i} \right)$$

In plainer language, the divergence is *the average difference in log probability between the target (p) and model (q)*. This divergence is just the difference between two entropies: The entropy of the target distribution p and the *cross entropy* arising from using q to predict p (see the Overthinking box on the next page for some more detail). When $p = q$, we know the actual probabilities of the events. In that case:

$$D_{\text{KL}}(p, q) = D_{\text{KL}}(p, p) = \sum_i p_i \big(\log(p_i) - \log(p_i) \big) = 0$$

There is no additional uncertainty induced when we use a probability distribution to represent itself. That's somehow a comforting thought.

But more importantly, as q grows more different from p, the divergence D_{KL} also grows. FIGURE 7.5 displays an example. Suppose the true target distribution is $p = \{0.3, 0.7\}$. Suppose the approximating distribution q can be anything from $q = \{0.01, 0.99\}$ to $q = \{0.99, 0.01\}$. The first of these probabilities, q_1, is displayed on the horizontal axis, and the

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

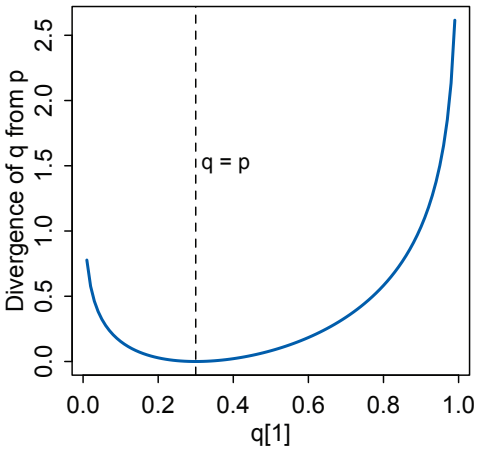


FIGURE 7.5. Information divergence of an approximating distribution q from a true distribution p. Divergence can only equal zero when $q = p$ (dashed line). Otherwise, the divergence is positive and grows as q becomes more dissimilar from p. When we have more than one candidate approximation q, the q with the smallest divergence is the most accurate approximation, in the sense that it induces the least additional uncertainty.

vertical displays the divergence $D_{\text{KL}}(p, q)$. Only exactly where $q = p$, at $q_1 = 0.3$, does the divergence achieve a value of zero. Everyplace else, it grows.

What divergence can do for us now is help us contrast different approximations to p. As an approximating function q becomes more accurate, $D_{\text{KL}}(p, q)$ will shrink. So if we have a pair of candidate distributions, then the candidate that minimizes the divergence will be closest to the target. Since predictive models specify probabilities of events (observations), we can use divergence to compare the accuracy of models.

Overthinking: Cross entropy and divergence. Deriving divergence is easier than you might think. The insight is in realizing that when we use a probability distribution q to predict events from another distribution p, this defines something known as *cross entropy*: $H(p, q) = -\sum_i p_i \log(q_i)$. The notion is that events arise according the the p's, but they are expected according to the q's, so the entropy is inflated, depending upon how different p and q are. Divergence is defined as the *additional* entropy induced by using q. So it's just the difference between $H(p)$, the actual entropy of events, and $H(p, q)$:

$$\begin{aligned} D_{\text{KL}}(p, q) &= H(p, q) - H(p) \\ &= -\sum_i p_i \log(q_i) - \big(-\sum_i p_i \log(p_i)\big) = -\sum_i p_i \big(\log(q_i) - \log(p_i)\big) \end{aligned}$$

So divergence really is measuring how far q is from the target p, in units of entropy. Notice that which is the target matters: $H(p, q)$ does not in general equal $H(q, p)$. For more on that fact, see the Rethinking box that follows.

Rethinking: Divergence depends upon direction. In general, $H(p, q)$ is not equal to $H(q, p)$. The direction matters, when computing divergence. Understanding why this is true is of some value, so here's a contrived teaching example.

Suppose we get in a rocket and head to Mars. But we have no control over our landing spot, once we reach Mars. Let's try to predict whether we land in water or on dry land, using the Earth to provide a probability distribution q to approximate the actual distribution on Mars, p. For the Earth, $q = \{0.7, 0.3\}$, for probability of water and land, respectively. Mars is very dry, but let's say for the sake of the example that there is 1% surface water, so $p = \{0.01, 0.99\}$. If we count the ice caps, that's not too big a lie. Now compute the divergence going from Earth to Mars. It turns out to be

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

$D_{\text{E}\to\text{M}} = D_{\text{KL}}(p, q) = 1.14$. That's the additional uncertainty induced by using the Earth to predict the Martian landing spot. Now consider going back the other direction. The numbers in p and q stay the same, but we swap their roles, and now $D_{\text{M}\to\text{E}} = D_{\text{KL}}(q, p) = 2.62$. The divergence is more than double in this direction. This result seems to defy comprehension. How can the distance from Earth to Mars be shorter than the distance from Mars to Earth?

Divergence behaves this way as a feature, not a bug. There really is more additional uncertainty induced by using Mars to predict Earth than by using Earth to predict Mars. The reason is that, going from Mars to Earth, Mars has so little water on its surface that we will be very very surprised when we most likely land in water on Earth. In contrast, Earth has good amounts of both water and dry land. So when we use the Earth to predict Mars, we expect both water and land, to some extent, even though we do expect more water than land. So we won't be nearly as surprised when we inevitably arrive on Martian dry land, because 30% of Earth is dry land.

An important practical consequence of this asymmetry, in a model fitting context, is that if we use a distribution with high entropy to approximate an unknown true distribution of events, we will reduce the distance to the truth and therefore the error. This fact will help us build generalized linear models, later on in Chapter 10.

7.2.4. Estimating divergence. At this point in the chapter, dear reader, you may be wondering where the chapter is headed. At the start, the goal was to deal with overfitting and underfitting. But now we've spent pages and pages on entropy and other fantasies. It's as if I promised you a day at the beach, but now you find yourself at a dark cabin in the woods, wondering if this is a necessary detour or rather a sinister plot.

It is a necessary detour. The point of all the preceding material about information theory and divergence is to establish both:

(1) How to measure the distance of a model from our target. Information theory gives us the distance measure we need, the KL divergence.

(2) How to estimate the divergence. Having identified the right measure of distance, we now need a way to estimate it in real statistical modeling tasks.

Item (1) is accomplished. Item (2) remains for last. You're going to see now that the divergence leads to using a measure of model fit known as *deviance*.

To use D_{KL} to compare models, it seems like we would have to know p, the target probability distribution. In all of the examples so far, I've just assumed that p is known. But when we want to find a model q that is the best approximation to p, the "truth," there is usually no way to access p directly. We wouldn't be doing statistical inference, if we already knew p.

But there's an amazing way out of this predicament. It helps that we are only interested in comparing the divergences of different candidates, say q and r. In that case, most of p just subtracts out, because there is a $\text{E}\log(p_i)$ term in the divergence of both q and r. This term has no effect on the distance of q and r from one another. So while we don't know where p is, we can estimate how far apart q and r are, and which is closer to the target. It's as if we can't tell how far any particular archer is from hitting the target, but we can tell which archer gets closer and by how much.

All of this also means that all we need to know is a model's average log-probability: $\text{E}\log(q_i)$ for q and $\text{E}\log(r_i)$ for r. These expressions look a lot like log-probabilities of outcomes you've been using already to simulate implied predictions of a fit model. Indeed, just summing the log-probabilities of each observed case provides an approximation of $\text{E}\log(q_i)$. We don't have to know the p inside the expectation.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

So we can compare the average log-probability from each model to get an estimate of the relative distance of each model from the target. This also means that the absolute magnitude of these values will not be interpretable—neither $\mathrm{E}\log(q_i)$ nor $\mathrm{E}\log(r_i)$ by itself suggests a good or bad model. Only the difference $\mathrm{E}\log(q_i) - \mathrm{E}\log(r_i)$ informs us about the divergence of each model from the target p.

To put all this into practice, it is conventional to sum over all the observations i, yielding a total score for a model q:

$$S(q) = \sum_i \log(q_i)$$

This kind of score is a log-probability score, and it is the gold standard way to compare the predictive accuracy of different models. It is an estimate of $\mathrm{E}\log(q_i)$, just without the final step of dividing by the number of observations.

To compute this score for a Bayesian model, we have to use the entire posterior distribution. Otherwise, vengeful angels will descend upon you. Why will they be angry? If we don't use the entire posterior, we are throwing away information. Because the parameters have distributions, the predictions also have a distribution. How can we use the entire distribution of predictions? We need to find the log of the average probability for each observation i, where the average is taken over the posterior distribution. Doing this calculation correctly requires a little subtlety. The `rethinking` package has a function called `lppd`—**LOG-POINTWISE-PREDICTIVE-DENSITY**—to do this calculation for `quap` models. If you are interested in the subtle details, however, see the box at the end of this section. To compute lppd for the first model we fit in this chapter:

R code 7.13

```
set.seed(1)
lppd( m7.1 , n=1e4 )
```

```
[1]  0.6098668  0.6483438  0.5496093  0.6234934  0.4648143  0.4347605 -0.8444633
```

Each of these values is the log-probability score for a specific observation. Recall that there were only 7 observations in those data. If you sum these values, you'll have the total log-probability score for the model and data. What do these values mean? Larger values are better, because that indicates larger average accuracy. It is also quite common to see something called the **DEVIANCE**, which is like a lppd score, but multiplied by -2 so that smaller values are better. The 2 is there for historical reasons.[112]

Overthinking: Computing the lppd. The Bayesian version of the log-probability score is called the **LOG-POINTWISE-PREDICTIVE-DENSITY**. For some data y and posterior distribution Θ:

$$\mathrm{lppd}(y, \Theta) = \sum_i \log \frac{1}{S} \sum_s p(y_i|\Theta_s)$$

where S is the number of samples and Θ_s is the s-th set of sampled parameter values in the posterior distribution. While in principle this is easy—you just need to compute the probability (density) of each observation i for each sample s, take the average, and then the logarithm—in practice it is not so easy. The reason is that doing arithmetic in a computer often requires some tricks to retain precision. In probability calculations, it is usually safest to do everything on the log-probability scale. Here's the code we need, to repeat the calculation in the previous section:

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

R code 7.14

```
set.seed(1)
logprob <- sim( m7.1 , ll=TRUE , n=1e4 )
n <- ncol(logprob)
ns <- nrow(logprob)
f <- function( i ) log_sum_exp( logprob[,i] ) - log(ns)
( lppd <- sapply( 1:n , f ) )
```

You should see the same values as before. The code first calculates the log-probability of each observation, using `sim`. You used `sim` in Chapter 4 to simulate observations from the posterior. It can also just return the log-probability, using `ll=TRUE`. It returns a matrix with a row for each sample and a column for each observation. Then the function `f` does the hard work. `log_sum_exp` computes the log of the sum of exponentiated values. So it takes all the log-probabilities for a given observation, exponentiates each, sums them, then takes the log. But it does this in a way that is numerically stable. Then the function subtracts the log of the number of samples, which is the same as dividing the sum by the number of samples.

7.2.5. Scoring the right data. The log-probability score is a principled way to measure distance from the target. But the score as computed in the previous section has the same flaw as R^2: It always improves as the model gets more complex, at least for the types of models we have considered so far. Just like R^2, log-probability on training data is a measure of retrodictive accuracy, not predictive accuracy. Let's compute the log-score for each of the models from earlier in this chapter:

R code 7.15

```
set.seed(1)
sapply( list(m7.1,m7.2,m7.3,m7.4,m7.5,m7.6) , function(m) sum(lppd(m)) )
```

```
[1]  2.490390  2.565982  3.695910  5.380871 14.089261 39.445390
```

The more complex models have larger scores! But we already know that they are absurd. We simply cannot score models by their performance on training data. That way lies the monster Scylla, devourer of naive data scientists.

It is really the score on new data that interests us. So before looking at tools for improving and measuring out-of-sample score, let's bring the problem into sharper focus by simulating the score both in and out of sample. When we usually have data and use it to fit a statistical model, the data comprise a **TRAINING SAMPLE**. Parameters are estimated from it, and then we can imagine using those estimates to predict outcomes in a new sample, called the **TEST SAMPLE**. R is going to do all of this for you. But here's the full procedure, in outline:

(1) Suppose there's a training sample of size N.
(2) Compute the posterior distribution of a model for the training sample, and compute the score on the training sample. Call this score D_{train}.
(3) Suppose another sample of size N from the same process. This is the test sample.
(4) Compute the score on the test sample, using the posterior trained on the training sample. Call this new score D_{test}.

The above is a thought experiment. It allows us to explore the distinction between accuracy measured in and out of sample, using a simple prediction scenario.

To visualize the results of the thought experiment, what we'll do now is conduct the above thought experiment 10,000 times, for each of five different linear regression models.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

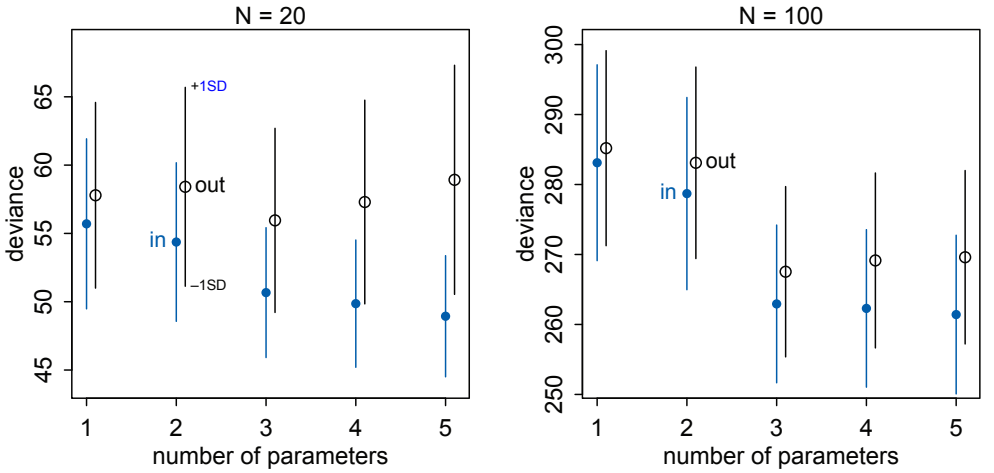


FIGURE 7.6. Deviance in and out of sample. In each plot, models with different numbers of predictor variables are shown on the horizontal axis. Deviance across 10,000 simulations is shown on the vertical. Blue shows deviance in-sample, the training data. Black shows deviance out-of-sample, the test data. Points show means, and the line segments show ±1 standard deviation.

The model that generates the data is:

$$y_i \sim \text{Normal}(\mu_i, 1)$$
$$\mu_i = (0.15)x_{1,i} - (0.4)x_{2,i}$$

This corresponds to a Gaussian outcome y for which the intercept is $\alpha = 0$ and the slopes for each of two predictors are $\beta_1 = 0.15$ and $\beta_2 = -0.4$. The models for analyzing the data are linear regressions with between 1 and 5 free parameters. The first model, with 1 free parameter to estimate, is just a linear regression with an unknown mean and fixed $\sigma = 1$. Each parameter added to the model adds a predictor variable and its beta-coefficient. Since the "true" model has non-zero coefficients for only the first two predictors, we can say that the true model has 3 parameters. By fitting all five models, with between 1 and 5 parameters, to training samples from the same processes, we can get an impression for how the score behaves, both inside and outside the training sample.

FIGURE 7.6 shows the results of 10,000 simulations for each model type, at two different sample sizes. The function that conducts the simulations is `sim_train_test` in the `rethinking` package. If you want to conduct more simulations of this sort, see the Overthinking box on the next page for the full code. The vertical axis is scaled as $-2 \times$ lppd, "deviance," so that larger values are worse. In the left-hand plot in FIGURE 7.6, both training and test samples contain 20 cases. Blue points and line segments show the mean plus-and-minus one standard deviation of the deviance calculated on the training data. Moving left to right with increasing numbers of parameters, the average deviance declines. A smaller deviance means a better fit. So this decline with increasing model complexity is the same phenomenon you saw earlier in the chapter with R^2.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

But now inspect the open points and black line segments. These display the distribution of out-of-sample deviance at each number of parameters. While the training deviance always gets better with an additional parameter, the test deviance is smallest on average for 3 parameters, which is the data-generating model in this case. The deviance out-of-sample gets worse (increases) with the addition of each parameter after the third. These additional parameters fit the noise in the additional predictors. So while deviance keeps improving (declining) in the training sample, it gets worse on average in the test sample. The right-hand plot shows the same relationships for larger samples of $N = 100$ cases.

The size of the standard deviation bars may surprise you. While it is always true on average that deviance out-of-sample is worse than deviance in-sample, any individual pair of train and test samples may reverse the expectation. The reason is that any given training sample may be highly misleading. And any given testing sample may be unrepresentative. Keep this fact in mind as we develop devices for comparing models, because this fact should prevent you from placing too much confidence in analysis of any particular sample. Like all of statistical inference, there are no guarantees here.

On that note, there is also no guarantee that the "true" data-generating model will have the smallest average out-of-sample deviance. You can see a symptom of this fact in the deviance for the 2 parameter model. That model does worse in prediction than the model with only 1 parameter, even though the true model does include the additional predictor. This is because with only $N = 20$ cases, the imprecision of the estimate for the first predictor produces more error than just ignoring it. In the right-hand plot, in contrast, there is enough data to precisely estimate the association between the first predictor and the outcome. Now the deviance for the 2 parameter model is better than that of the 1 parameter model.

Deviance is an assessment of predictive accuracy, not of truth. The true model, in terms of which predictors are included, is not guaranteed to produce the best predictions. Likewise a false model, in terms of which predictors are included, is not guaranteed to produce poor predictions.

The point of this thought experiment is to demonstrate how deviance behaves, in theory. While deviance on training data always improves with additional predictor variables, deviance on future data may or may not, depending upon both the true data-generating process and how much data is available to precisely estimate the parameters. These facts form the basis for understanding both regularizing priors and information criteria.

Overthinking: Simulated training and testing. To reproduce Figure 7.6, `sim.train.test` is run 10,000 (`1e4`) times for each of the 5 models. This code is sufficient to run all of the simulations:

R code 7.16

```
N <- 20
kseq <- 1:5
dev <- sapply( kseq , function(k) {
        print(k);
        r <- replicate( 1e4 , sim_train_test( N=N, k=k ) );
        c( mean(r[1,]) , mean(r[2,]) , sd(r[1,]) , sd(r[2,]) )
    } )
```

If you use Mac OS or Linux, you can parallelize the simulations by replacing the `replicate` line with:

R code 7.17

```
        r <- mcreplicate( 1e4 , sim_train_test( N=N, k=k ) , mc.cores=4 )
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Set `mc.cores` to the number of processor cores you want to use for the simulations. Once the simulations complete, `dev` will be a 4-by-5 matrix of means and standard deviations. To reproduce the plot:

R code 7.18

```
plot( 1:5 , dev[1,] , ylim=c( min(dev[1:2,])-5 , max(dev[1:2,])+10 ) ,
    xlim=c(1,5.1) , xlab="number of parameters" , ylab="deviance" ,
    pch=16 , col=rangi2 )
mtext( concat( "N = ",N ) )
points( (1:5)+0.1 , dev[2,] )
for ( i in kseq ) {
    pts_in <- dev[1,i] + c(-1,+1)*dev[3,i]
    pts_out <- dev[2,i] + c(-1,+1)*dev[4,i]
    lines( c(i,i) , pts_in , col=rangi2 )
    lines( c(i,i)+0.1 , pts_out )
}
```

By altering this code, you can simulate many different train-test scenarios. See `?sim_train_test` for additional options.

7.3. Golem taming: regularization

What if I told you that one way to produce better predictions is to make the model worse at fitting the sample? Would you believe it? In this section, we'll demonstrate it.

The root of overfitting is a model's tendency to get overexcited by the training sample. When the priors are flat or nearly flat, the machine interprets this to mean that every parameter value is equally plausible. As a result, the model returns a posterior that encodes as much of the training sample—as represented by the likelihood function—as possible.

One way to prevent a model from getting too excited by the training sample is to use a skeptical prior. By "skeptical," I mean a prior that slows the rate of learning from the sample. The most common skeptical prior is a **REGULARIZING PRIOR**. Such a prior, when tuned properly, reduces overfitting while still allowing the model to learn the regular features of a sample. If the prior is too skeptical, however, then regular features will be missed, resulting in underfitting. So the problem is really one of tuning. But as you'll see, even mild skepticism can help a model do better, and doing better is all we can really hope for in the large world, where no model nor prior is optimal.

In previous chapters, I forced us to revise the priors until the prior predictive distribution produced only reasonable outcomes. As a consequence, those priors regularized inference. In very small samples, they would be a big help. Here I want to show you why, using some more simulations. Consider this Gaussian model:

$$
\begin{aligned}
y_i &\sim \text{Normal}(\mu_i, \sigma) \\
\mu_i &= \alpha + \beta x_i \\
\alpha &\sim \text{Normal}(0, 100) \\
\beta &\sim \text{Normal}(0, 1) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

Assume, as is good practice, that the predictor x is standardized so that its standard deviation is 1 and its mean is zero. Then the prior on α is a nearly flat prior that has no practical effect on inference, as you've seen in earlier chapters.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

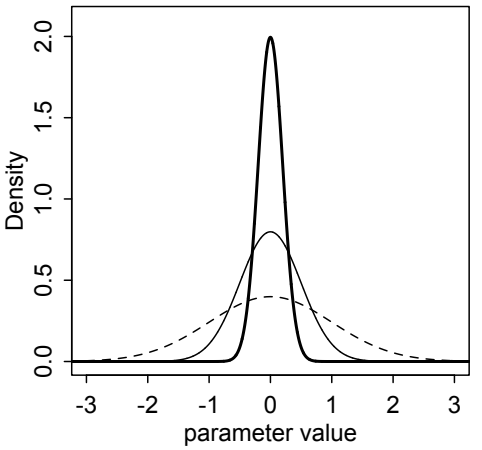


FIGURE 7.7. Regularizing priors, weak and strong. Three Gaussian priors of varying standard deviation. These priors reduce overfitting, but with different strength. Dashed: Normal$(0, 1)$. Thin solid: Normal$(0, 0.5)$. Thick solid: Normal$(0, 0.2)$.

But the prior on β is narrower and is meant to regularize. The prior $\beta \sim \text{Normal}(0, 1)$ says that, before seeing the data, the machine should be very skeptical of values above 2 and below -2, as a Gaussian prior with a standard deviation of 1 assigns only 5% plausibility to values above and below 2 standard deviations. Because the predictor variable x is standardized, you can interpret this as meaning that a change of 1 standard deviation in x is very unlikely to produce 2 units of change in the outcome.

You can visualize this prior in FIGURE 7.7 as the dashed curve. Since more probability is massed up around zero, estimates are shrunk towards zero—they are conservative. The other curves are narrower priors that are even more skeptical of parameter values far from zero. The thin solid curve is a stronger Gaussian prior with a standard deviation of 0.5. The thick solid curve is even stronger, with a standard deviation of only 0.2.

How strong or weak these skeptical priors will be in practice depends upon the data and model. So let's explore a train-test example, similar to what you saw in the previous section (FIGURE 7.6). This time we'll use the regularizing priors pictured in FIGURE 7.7, instead of flat priors. For each of five different models, we simulate 10,000 times for each of the three regularizing priors above. FIGURE 7.8 shows the results. The points are the same flat-prior deviances as in the previous section: blue for training deviance and black for test deviance. The lines show the train and test deviances for the different priors. The blue lines are training deviance and the black lines test deviance. The style of the lines correspond to those in FIGURE 7.7.

Focus on the left-hand plot, where the sample size is $N = 20$, for the moment. The training deviance always increases—gets worse—with tighter priors. The thick blue trend is substantially larger than the others, and this is because the skeptical prior prevents the model from adapting completely to the sample. But the test deviances, out-of-sample, improve (get smaller) with the tighter priors. The model with three parameters is still the best model out-of-sample, and the regularizing priors have little impact on its deviance.

But also notice that as the prior gets more skeptical, the harm done by an overly complex model is greatly reduced. For the Normal$(0, 0.2)$ prior (thick line), the models with 4 and 5 parameters are barely worse than the correct model with 3 parameters. If you can tune the regularizing prior right, then overfitting can be greatly reduced.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

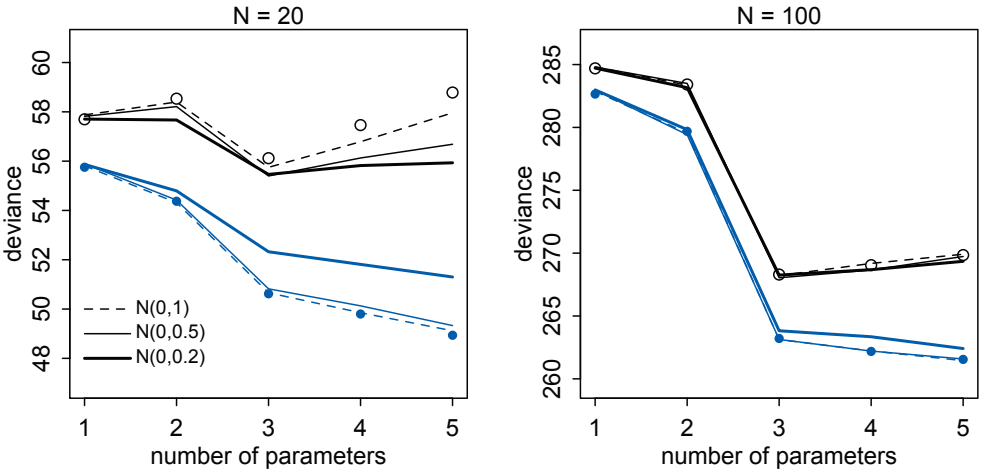


FIGURE 7.8. Regularizing priors and out-of-sample deviance. The points in both plots are the same as in FIGURE 7.6. The lines show training (blue) and testing (black) deviance for the three regularizing priors in FIGURE 7.7. Dashed: Each beta-coefficient is given a Normal$(0, 1)$ prior. Thin solid: Normal$(0, 0.5)$. Thick solid: Normal$(0, 0.2)$.

Now focus on the right-hand plot, where sample size is $N = 100$. The priors have much less of an effect here, because there is so much more evidence. The priors do help. But overfitting was less of a concern to begin with, and there is enough information in the data to overwhelm even the Normal$(0, 0.2)$ prior (thick line).

Regularizing priors are great, because they reduce overfitting. But if they are too skeptical, they prevent the model from learning from the data. When you encounter multilevel models in Chapter 13, you'll see that their central device is to learn the strength of the prior from the data itself. So you can think of multilevel models as adaptive regularization, where the model itself tries to learn how skeptical it should be.

Rethinking: Ridge regression. Linear models in which the slope parameters use Gaussian priors, centered at zero, are sometimes known as **RIDGE REGRESSION**. Ridge regression typically takes as input a precision λ that essentially describes the narrowness of the prior. $\lambda > 0$ results in less overfitting. However, just as with the Bayesian version, if λ is too large, we risk underfitting. While not originally developed as Bayesian, ridge regression is another example of how a statistical procedure can be understood from both Bayesian and non-Bayesian perspectives. Ridge regression does not compute a posterior distribution. Instead it uses a modification of OLS that stitches λ into the usual matrix algebra formula for the estimates. The function `lm.ridge`, built into R's `MASS` library, will fit linear models this way.

Despite how easy it is to use regularization, most traditional statistical methods use no regularization at all. Statisticians often make fun of machine learning for reinventing statistics under new names. But regularization is one area where machine learning is more mature. Introductory machine learning courses usually describe regularization. Most introductory statistics courses do not.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

7.4. Predicting predictive accuracy

All of the preceding suggests one way to navigate overfitting and underfitting: Evaluate our models out-of-sample. But we do not have the out-of-sample, by definition, so how can we evaluate our models on it? There are two families of strategies: **CROSS-VALIDATION** and **INFORMATION CRITERIA**. These strategies try to guess how well models will perform, on average, in predicting new data. We'll consider both approaches in more detail. Despite subtle differences in their mathematics, they produce extremely similar approximations.

7.4.1. Cross-validation. A popular strategy for estimating predictive accuracy is to actually test the model's predictive accuracy on another sample. This is known as **CROSS-VALIDATION**, leaving out a small chunk of observations from our sample and evaluating the model on the observations that were left out. Of course we don't want to leave out data. So what is usually done is to divide the sample in a number of chunks, called "folds." The model is asked to predict each fold, after training on all the others. We then average over the score for each fold to get an estimate of out-of-sample accuracy. The minimum number of folds is 2. At the other extreme, you could make each point observation a fold and fit as many models as you have individual observations. You can perform cross-validation on `quap` models using the `cv_quap` function in the `rethinking` package.

How many folds should you use? This is an understudied question. A lot of advice states that both too few and too many folds produce less reliable approximations of out-of-sample performance. But simulation studies do not reliably find that this is the case.[113] It is extremely common to use the maximum number of folds, resulting in leaving out one unique observation in each fold. This is called **LEAVE-ONE-OUT CROSS-VALIDATION** (often abbreviated as LOOCV). Leave-one-out cross-validation is what we'll consider in this chapter, and it is the default in `cv_quap`.

The key trouble with leave-one-out cross-validation is that, if we have 1000 observations, that means computing 1000 posterior distributions. That can be time consuming. Luckily, there are clever ways to approximate the cross-validation score without actually running the model over and over again. One approach is to use the "importance" of each observation to the posterior distribution. What "importance" means here is that some observations have a larger impact on the posterior distribution—if we remove an important observation, the posterior changes more. Other observations have less impact. It is a benign aspect of the universe that this importance can be estimated without refitting the model.[114] The key intuition is that an observation that is relatively unlikely is more important than one that is relatively expected. When your expectations are violated, you should change your expectation more. Bayesian inference works the same way. This importance is often called a *weight*, and these weights can be used to estimate a model's out-of-sample accuracy.

Smuggling a bunch of mathematical details under the carpet, this strategy results in a useful approximation of the cross-validation score. The approximation goes by the awkward name of **PARETO-SMOOTHED IMPORTANCE SAMPLING CROSS-VALIDATION**.[115] We'll call it **PSIS** for short, and the `PSIS` function will compute it. PSIS uses importance sampling, which just means that it uses the importance weights approach described in the previous paragraph. The Pareto-smoothing is a technique for making the importance weights more reliable. Pareto is the name of a small town in northern Italy. But it is also the name of an Italian scientist, Vilfredo Pareto (1848–1923), who made many important contributions. One of these is known as the **PARETO DISTRIBUTION**. PSIS uses this distribution to derive

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

more reliable cross-validation score, without actually doing any cross-validation. If you want a little more detail, see the Overthinking box below.

The best feature of PSIS is that it provides feedback about its own reliability. It does this by noting particular observations with very high weights that could make the PSIS score inaccurate. We'll look at this in much more detail both later in this chapter and in several examples in the remainder of the book.

Another nice feature of cross-validation and PSIS as an approximation is that it is computed point by point. This pointwise nature provides an approximate—sometimes very approximate—estimate of the standard error of our estimate of out-of-sample deviance. To compute this standard error, we calculate the CV or PSIS score for each observation and then exploit the central limit theorem to provide a measure of the standard error:

$$s_{\text{PSIS}} = \sqrt{N \text{var}(\text{psis}_i)}$$

where N is the number of observations and psis_i is the PSIS estimate for observation i. If this doesn't quite make sense, be sure to look at the code box at the end of this section (page 222).

Overthinking: Pareto-smoothed cross-validation. Cross-validation estimates the out-of-sample LOG-POINTWISE-PREDICTIVE-DENSITY (lppd, page 210). If you have N observations and fit the model N times, dropping a single observation y_i each time, then the out-of-sample lppd is the sum of the average accuracy for each omitted y_i.

$$\text{lppd}_{\text{CV}} = \sum_{i=1}^{N} \frac{1}{S} \sum_{s=1}^{S} \log \Pr(y_i|\theta_{-i,s})$$

where s indexes samples from a Markov chain and $\theta_{-i,s}$ is the s-th sample from the posterior distribution computed for observations omitting y_i.

Importance sampling replaces the computation of N posterior distributions by using an estimate of the importance of each i to the posterior distribution. We draw samples from the full posterior distribution $p(\theta|y)$, but we want samples from the reduced leave-one-out posterior distribution $p(\theta|y_{-i})$. So we re-weight each sample s by the inverse of the probability of the omitted observation:[116]

$$r(\theta_s) = \frac{1}{p(y_i|\theta_s)}$$

This weight is only relative, but it is normalized inside the calculation like this:

$$\text{lppd}_{\text{IS}} = \sum_{i=1}^{N} \log \frac{\sum_{s=1}^{S} r(\theta_s) p(y_i|\theta_s)}{\sum_{s=1}^{S} r(\theta_s)}$$

And that is the importance sampling estimate of out-of-sample lppd.

We haven't done any Pareto smoothing yet, however. The reason we need to is that the weights $r(\theta_s)$ can be unreliable. In particular, if any $r(\theta_s)$ is too relatively large, it can ruin the estimate of lppd by dominating it. One strategy is to truncate the weights so that none are larger than a theoretically derived limit. This helps, but it also biases the estimate. What PSIS does is more clever. It exploits the fact that the distribution of weights should have a particular shape, under some regular conditions. The largest weights should follow a generalized PARETO DISTRIBUTION:

$$p(r|u,\sigma,k) = \sigma^{-1}\left(1 + k(r-u)\sigma^{-1}\right)^{-\frac{1}{k}-1}$$

where u is the location parameter, σ is the scale, and k is the shape. For each observation y_i, the largest weights are used to estimate a Pareto distribution and then smoothed using that Pareto distribution. This works quite well, both in theory and practice.[117] The best thing about the approach however is that the estimates of k provide information about the reliability of the approximation. There will be one k value for each y_i. Larger k values indicate more influential points, and if $k > 0.5$, then the

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Pareto distribution has infinite variance. A distribution with infinite variance has a very thick tail. Since we are trying to smooth the importance weights with the distribution's tail, an infinite variance makes the weights harder to trust. Still, both theory and simulation suggest PSIS's weights perform well as long as $k < 0.7$. When we start using PSIS, you'll see warnings about large k values. These are very useful for identifying influential observations.

7.4.2. Information criteria. The second approach is the use of **INFORMATION CRITERIA** to compute an expected score out of sample. Information criteria construct a theoretical estimate of the relative out-of-sample KL divergence.

If you look back at FIGURE 7.8, there is a curious pattern in the distance between the points (showing the train-test pairs with flat priors): The difference is approximately twice the number of parameters in each model. The difference between training deviance and testing deviance is almost exactly 2 for the first model (with 1 parameter) and about 10 for the last (with 5 parameters). This is not a coincidence but rather one of the coolest results in machine learning: For ordinary linear regressions with flat priors, the expected overfitting penalty is about twice the number of parameters.

This is the phenomenon behind **INFORMATION CRITERIA**. The best known information criterion is the **AKAIKE INFORMATION CRITERION**, abbreviated **AIC**.[118] AIC provides a surprisingly simple estimate of the average out-of-sample deviance:

$$\text{AIC} = D_{\text{train}} + 2p = -2\text{lppd} + 2p$$

where p is the number of free parameters in the posterior distribution. As the 2 is just there for scaling, what AIC tells us is that the dimensionality of the posterior distribution is a natural measure of the model's overfitting tendency. More complex models tend to overfit more, directly in proportion to the number of parameters.

AIC is of mainly historical interest now. Newer and more general approximations exist that dominate AIC in every context. But Akaike deserves tremendous credit for the initial inspiration. See the box further down for more details. AIC is an approximation that is reliable only when:

(1) The priors are flat or overwhelmed by the likelihood.
(2) The posterior distribution is approximately multivariate Gaussian.
(3) The sample size N is much greater[119] than the number of parameters k.

Since flat priors are hardly ever the best priors, we'll want something more general. And when you get to multilevel models, the priors are never flat by definition. There is a more general criterion, the **DEVIANCE INFORMATION CRITERION** (DIC). DIC is okay with informative priors, but still assumes that the posterior is multivariate Gaussian and that $N \gg k$.[120]

Overthinking: The Akaike inspiration criterion. The Akaike Information Criterion is a truly elegant result. Hirotugu Akaike (赤池弘次, 1927–2009) explained how the insight came to him: "On the morning of March 16, 1971, while taking a seat in a commuter train, I suddenly realized that the parameters of the factor analysis model were estimated by maximizing the likelihood and that the mean value of the logarithmus of the likelihood was connected with the Kullback-Leibler information number."[121] Must have been some train. What was at the heart of Akaike's realization? Mechanically, deriving AIC means writing down the goal, which is the expected KL divergence, and then making approximations. The expected bias turns out to be proportional to the number of parameters, provided a number of assumptions are approximately correct.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

We'll focus on a criterion that is more general than both AIC and DIC. Sumio Watanabe's (渡辺澄夫) **Widely Applicable Information Criterion** (WAIC) makes no assumption about the shape of the posterior.[122] It provides an approximation of the out-of-sample deviance that converges to the cross-validation approximation in a large sample. But in a finite sample, it can disagree. It can disagree because it has a different target—it isn't trying to approximate the cross-validation score, but rather guess the out-of-sample KL divergence. In the large-sample limit, these tend to be the same.

How do we compute WAIC? Unfortunately, it's generality comes at the expense of a more complicated formula. But really it just has two pieces, and you can compute both directly from samples from the posterior distribution. WAIC is just the log-posterior-predictive-density (lppd, page 210) that we calculated earlier plus a penalty proportional to the variance in the posterior predictions:

$$\text{WAIC}(y,\Theta) = -2\Big(\text{lppd} - \underbrace{\sum_i \text{var}_\theta \log p(y_i|\theta)}_{\text{penalty term}}\Big)$$

where y is the observations and Θ is the posterior distribution. The penalty term means, "compute the variance in log-probabilities for each observation i, and then sum up these variances to get the total penalty." So you can think of each observation as having its own personal penalty score. And since these scores measure overfitting risk, you can also assess overfitting risk at the level of each observation.

Because of the analogy to Akaike's original criterion, the penalty term in WAIC is sometimes called the **effective number of parameters**, labeled p_{WAIC}. This label makes historical sense, but it doesn't make much mathematical sense. As we'll see as the book progresses, the overfitting risk of a model has less to do with the number of parameters than with how the parameters are related to one another. When we get to multilevel models, adding parameters to the model can actually *reduce* the "effective number of parameters." Like English language spelling, the field of statistics is full of historical baggage that impedes learning. No one chose this situation. It's just cultural evolution. I'll try to call the penalty term "the overfitting penalty." But if you see it called the effective number of parameters elsewhere, you'll know it is the same thing.

The function `WAIC` in the `rethinking` package will compute WAIC for a model fit with `quap` or `ulam` or `rstan` (which we'll use later in the book). If you want to see a didactic implementation of computing lppd and the penalty term, see the Overthinking box at the end of this section. Seeing the mathematical formula above as computer code may be what you need to understand it.

Like PSIS, WAIC is *pointwise*. Prediction is considered case-by-case, or point-by-point, in the data. Several things arise from this. First, WAIC also has an approximate standard error (see calculation in the Overthinking box on page 222). Second, since some observations have stronger influence on the posterior distribution, WAIC notes this in its pointwise penalty terms. Third, just like cross-validation and PSIS, because WAIC allows splitting up the data into independent observations, it is sometimes hard to define. Consider for example a model in which each prediction depends upon a previous observation. This happens, for example, in a *time series*. In a time series, a previous observation becomes a predictor variable for the next observation. So it's not easy to think of each observation as independent or *exchangeable*. In such a case, you can of course compute WAIC as if each observation were independent of the others, but it's not clear what the resulting value means.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

This caution raises a more general issue with all strategies to guess out-of-sample accuracy: Their validity depends upon the predictive task you have in mind. And not all prediction can reasonably take the form that we've been assuming for the train-test simulations in this chapter. When we consider multilevel models, this issue will arise again.

Rethinking: Information criteria and consistency. As mentioned previously, information criteria like AIC and WAIC do not always assign the best expected D_{test} to the "true" model. In statistical jargon, information criteria are not **consistent** for model identification. These criteria aim to nominate the model that will produce the best predictions, as judged by out-of-sample deviance, so it shouldn't surprise us that they do not also do something that they aren't designed to do. Other metrics for model comparison are however consistent. So are information criteria broken?

They are not broken, if you care about prediction.[123] Issues like consistency are nearly always evaluated *asymptotically*. This means that we imagine the sample size N approaching infinity. Then we ask how a procedure behaves in this large-data limit. With practically infinite data, AIC and WAIC and cross-validation will often select a more complex model, so they are sometimes accused of "overfitting." But at the large-data limit, the most complex model will make predictions identical to the true model (assuming it exists in the model set). The reason is that with so much data every parameter can be very precisely estimated. And so using an overly complex model will not hurt prediction. For example, as sample size $N \rightarrow \infty$ the model with 5 parameters in Figure 7.8 will tell you that the coefficients for predictors after the second are almost exactly zero. Therefore failing to identify the "correct" model does not hurt us, at least not in this sense. Furthermore, in the natural and social sciences the models under consideration are almost never the data-generating models. It makes little sense to attempt to identify a "true" model.

Rethinking: What about BIC and Bayes factors? The **Bayesian information criterion**, abbreviated BIC and also known as the Schwarz criterion,[124] is more commonly juxtaposed with AIC. The choice between BIC or AIC (or neither!) is not about being Bayesian or not. There are both Bayesian and non-Bayesian ways to motivate both, and depending upon how strict one wishes to be, neither is Bayesian. BIC is related to the logarithm of the *average likelihood* of a linear model. The average likelihood is the denominator in Bayes' theorem, the likelihood averaged over the prior. There is a venerable tradition in Bayesian inference of comparing average likelihoods as a means to comparing models. A ratio of average likelihoods is called a **Bayes factor**. On the log scale, these ratios are differences, and so comparing differences in average likelihoods resembles comparing differences in information criteria. Since average likelihood is averaged over the prior, more parameters induce a natural penalty on complexity. This helps guard against overfitting, even though the exact penalty is not the same as with information criteria.

Many Bayesian statisticians dislike the Bayes factor approach,[125] and all admit that there are technical obstacles to its use. One problem is that computing average likelihood is hard. Even when you can compute the posterior, you may not be able to estimate the average likelihood. Another problem is that, even when priors are weak and have little influence on posterior distributions within models, priors can have a huge impact on comparisons between models.

It's important to realize, though, that the choice of Bayesian or not does not also decide between information criteria or Bayes factors. Moreover, there's no need to choose, really. We can always use both and learn from the ways they agree and disagree. And both information criteria and Bayes factors are purely predictive criteria that will happily select confounded models. They know nothing about causation.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Overthinking: WAIC calculations. To see how the WAIC calculations actually work, consider a simple regression fit with quap:

R code 7.19
```
data(cars)
m <- quap(
    alist(
        dist ~ dnorm(mu,sigma),
        mu <- a + b*speed,
        a ~ dnorm(0,100),
        b ~ dnorm(0,10),
        sigma ~ dexp(1)
    ) , data=cars )
set.seed(94)
post <- extract.samples(m,n=1000)
```

We'll need the log-likelihood of each observation *i* at each sample *s* from the posterior:

R code 7.20
```
n_samples <- 1000
logprob <- sapply( 1:n_samples ,
    function(s) {
        mu <- post$a[s] + post$b[s]*cars$speed
        dnorm( cars$dist , mu , post$sigma[s] , log=TRUE )
    } )
```

You end up with a 50-by-1000 matrix of log-likelihoods, with observations in rows and samples in columns. Now to compute lppd, the Bayesian deviance, we average the samples in each row, take the log, and add all of the logs together. However, to do this with precision, we need to do all of the averaging on the log scale. This is made easy with a function `log_sum_exp`, which computes the log of a sum of exponentiated terms. Then we can just subtract the log of the number of samples. This computes the log of the average.

R code 7.21
```
n_cases <- nrow(cars)
lppd <- sapply( 1:n_cases , function(i) log_sum_exp(logprob[i,]) - log(n_samples) )
```

Typing `sum(lppd)` will give you lppd, as defined in the main text. Now for the penalty term, p_{WAIC}. This is more straightforward, as we just compute the variance across samples for each observation, then add these together:

R code 7.22
```
pWAIC <- sapply( 1:n_cases , function(i) var(logprob[i,]) )
```

And `sum(pWAIC)` returns p_{WAIC}, as defined in the main text. To compute WAIC:

R code 7.23
```
-2*( sum(lppd) - sum(pWAIC) )
```

```
[1] 423.3154
```

Compare to the output of the `WAIC` function. There will be simulation variance, because of how the samples are drawn from the `quap` fit. But that variance remains much smaller than the standard error of WAIC itself. You can compute the standard error by computing the square root of number of cases multiplied by the variance over the individual observation terms in WAIC:

R code 7.24
```
waic_vec <- -2*( lppd - pWAIC )
sqrt( n_cases*var(waic_vec) )
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

```
[1] 17.81628
```

As models get more complicated, all that usually changes is how the log-probabilities, `logprob`, are computed.

Note that each individual observation has its own penalty term in the `pWAIC` vector we calculated above. This provides an interesting opportunity to study how different observations contribute to overfitting. You can get the same vectorized pointwise output from the `WAIC` function by using the `pointwise=TRUE` argument.

7.4.3. Comparing CV, PSIS, and WAIC. With definitions of cross-validation, PSIS, and WAIC in hand, let's conduct another simulation exercise. This will let us visualize the estimates of out-of-sample deviance that these criteria provide, in the same familiar context as earlier sections. Our interest for now is in seeing how well the criteria approximate out-of-sample accuracy. Can they guess the overfitting risk?

FIGURE 7.9 shows the results of 1000 simulations each for the five familiar models with between 1 and 5 parameters, simulated under two different sets of priors and two different sample sizes. The plot is complicated. But taking it one piece at a time, all the parts are already familiar. Focus for now just on the top-left plot, where $N = 20$. The vertical axis is the out-of-sample deviance ($-2 \times$ lppd). The open points show the average out-of-sample deviance for models fit with flat priors. The filled points show the average out-of-sample deviance for models fit with regularizing priors with a standard deviation of 0.5. Notice that the regularizing priors overfit less, just as you saw in the previous section about regularizing priors. So that isn't new.

We are interested now in how well CV, PSIS, and WAIC approximate these points. Still focusing on the top-left plot in FIGURE 7.9, there are trend lines for each criterion. Solid black trends show WAIC. Solid blue trends show full cross-validation, computed by fitting the model N times. The dashed blue trends are PSIS. Notice that all three criteria do a good job of guessing the average out-of-sample score, whether the models used flat (upper trends) or regularizing (lower trends) priors. Provided the process generating data remains the same, it really is possible to use a single sample to guess the accuracy of our predictions.

While all three criteria get the expected out-of-sample deviance approximately correct, it is also true that in any particular sample they usually miss it by some amount. So we should look at the average error as well. The upper-right plot makes the average error of each measure easier to see. Now the vertical axis is the average absolute difference between the out-of-sample deviance and each criterion. WAIC (black trend) is slightly better on average. The bottom row repeats these plots for a larger sample size, $N = 100$. With a sample this large, in a family of models this simple, all three criteria become identical.

PSIS and WAIC perform very similarly in the context of ordinary linear models.[126] If there are important differences, they lie in other model types, where the posterior distribution is not approximately Gaussian or in the presence of observations that strongly influence the posterior. CV and PSIS have higher variance as estimators of the KL divergence, while WAIC has greater bias. So we should expect each to be slightly better in different contexts.[127] However, in practice any advantage may be much smaller than the expected error. Watanabe recommends computing both WAIC and PSIS and contrasting them. If there are large differences, this implies one or both criteria are unreliable.

Estimation aside, PSIS has a distinct advantage in warning the user about when it is unreliable. The k values that PSIS computes for each observation indicate when the PSIS

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

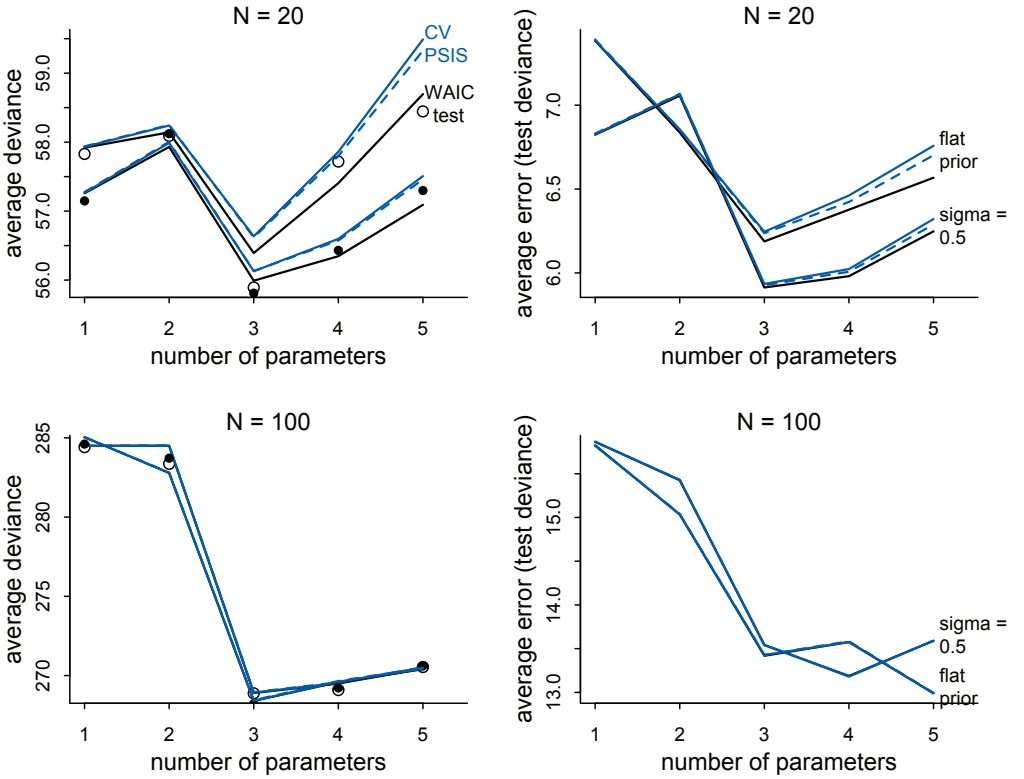


FIGURE 7.9. WAIC and cross-validation as estimates of the out-of-sample deviance. The top row displays 1000 train-test simulations with $N = 20$. The bottom row shows 1000 simulations with $N = 1000$. In each plot, there are two sets of trends. The open points are unregularized. The filled points are for regularizing $\sigma = 0.5$ priors. Left: The vertical axis is absolute deviance. Points are the average test deviance. The black line is the average WAIC estimate. Blue is the leave-one-out cross-validation (CV) score, and dashed blue is the PSIS approximation of the cross-validation score. Right: The same data, but now shown on the scale of average error in approximating the test deviance.

score may be unreliable, as well as identify which observations are at fault. We'll see later how useful this can be.

Rethinking: Diverse prediction frameworks. The train-test gambit we've been using in this chapter entails predicting a test sample of the same size and nature as the training sample. This most certainly does not mean that information criteria can only be used when we plan to predict a sample of the same size as training. The same size just scales the out-of-sample deviance similarly. It is the distance between the models that is useful, not the absolute value of the deviance. Nor do cross-validation and information criteria require that the data generating model be one of the models being considered. That was true in our simulations. But it isn't a requirement for them to help in identifying good models for prediction.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

But the train-test prediction task is not representative of everything we might wish to do with models. For example, some statisticians prefer to evaluate predictions using a **PREQUENTIAL** framework, in which models are judged on their accumulated learning error over the training sample.[128] And once you start using multilevel models, "prediction" is no longer uniquely defined, because the test sample can differ from the training sample in ways that forbid use of some the parameter estimates. We'll worry about that issue in Chapter 13.

Perhaps a larger concern is that our train-test thought experiment pulls the test sample from exactly the same process as the training sample. This is a kind of *uniformitarian* assumption, in which future data are expected to come from the same process as past data and have the same rough range of values. This can cause problems. For example, suppose we fit a regression that predicts height using body weight. The training sample comes from a poor town, in which most people are pretty thin. The relationship between height and weight turns out to be positive and strong. Now also suppose our prediction goal is to guess the heights in another, much wealthier, town. Plugging the weights from the wealthy individuals into the model fit to the poor individuals will predict outrageously tall people. The reason is that, once weight becomes large enough, it has essentially no relationship with height. WAIC will not automatically recognize nor solve this problem. Nor will any other isolated procedure. But over repeated rounds of model fitting, attempts at prediction, and model criticism, it is possible to overcome this kind of limitation. As always, statistics is no substitute for science.

7.5. Model comparison

Let's review the original problem and the road so far. When there are several plausible (and hopefully un-confounded) models for the same set of observations, how should we compare the accuracy of these models? Following the fit to the sample is no good, because fit will always favor more complex models. Information divergence is the right measure of model accuracy, but even it will just lead us to choose more and more complex and wrong models. We need to somehow evaluate models out-of-sample. How can we do that? A metamodel of forecasting tells us two important things. First, flat priors produce bad predictions. Regularizing priors—priors which are skeptical of extreme parameter values—reduce fit to sample but tend to improve predictive accuracy. Second, we can get a useful guess of predictive accuracy with the criteria CV, PSIS, and WAIC. Regularizing priors and CV/PSIS/WAIC are complementary. Regularization reduces overfitting, and predictive criteria measure it.

That's the road so far, the conceptual journey. And that's the hardest part. Using tools like PSIS and WAIC is much easier than understanding them. Which makes them quite dangerous. That is why this chapter has spent so much time on foundations, without doing any actual data analysis.

Now let's do some analysis. How do we use regularizing priors and CV/PSIS/WAIC? A very common use of cross-validation and information criteria is to perform **MODEL SELECTION**, which means choosing the model with the lowest criterion value and then discarding the others. But you should never do this. This kind of selection procedure discards the information about relative model accuracy contained in the differences among the CV/PSIS/WAIC values. Why are the differences useful? Because sometimes the differences are large and sometimes they are small. Just as relative posterior probability provides advice about how confident we might be about parameters (conditional on the model), relative model accuracy provides advice about how confident we might be about models (conditional on the set of models compared).

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Another reason to never select models based upon WAIC/CV/PSIS alone is that we might care about causal inference. Maximizing expected predictive accuracy is not the same as inferring causation. Highly confounded models can still make good predictions, at least in the short term. They won't tell us the consequences of an intervention, but they might help us forecast. So we need to be clear about our goals and not just toss variables into the causal salad and let WAIC select our meal.

So what good are these criteria then? They measure expected predictive value of a variable on the right scale, accounting for overfitting. This helps in testing model implications, given a set of causal models. They also provide a way to measure the overfitting tendency of a model, and that helps us both design models and understand how statistical inference works. Finally, minimizing a criterion like WAIC can help in designing models, especially in tuning parameters in multilevel models.

So instead of model *selection*, we'll focus on **MODEL COMPARISON**. This is a more general approach that uses multiple models to understand both how different variables influence predictions and, in combination with a causal model, implied conditional independencies among variables help us infer causal relationships.

We'll work through two examples. The first emphasizes the distinction between comparing models for predictive performance versus comparing them in order to infer causation. The second emphasizes the pointwise nature of model comparison and what inspecting individual points can reveal about model performance and mis-specification. This second example also introduces a more robust alternative to Gaussian regression.

7.5.1. Model mis-selection. We must keep in mind the lessons of the previous chapters: Inferring cause and making predictions are different tasks. Cross-validation and WAIC aim to find models that make good predictions. They don't solve any causal inference problem. If you select a model based only on expected predictive accuracy, you could easily be confounded. The reason is that backdoor paths do give us valid information about statistical associations in the data. So they can improve prediction, as long as we don't intervene in the system and the future is like the past. But recall that our working definition of knowing a cause is that we can predict the consequences of an intervention. So a good PSIS or WAIC score does not in general indicate a good causal model.

For example, recall the plant growth example from the previous chapter. The model that conditions on fungus will make better predictions than the model that omits it. If you return to that section (page 171) and run models `m6.6`, `m6.7`, and `m6.8` again, we can compare their WAIC values. To remind you, `m6.6` is the model with just an intercept, `m6.7` is the model that includes both treatment and fungus (the post-treatment variable), and `m6.8` is the model that includes treatment but omits fungus. It's `m6.8` that allows us to correctly infer the causal influence of treatment.

To begin, let's use the `WAIC` convenience function to calculate WAIC for `m6.7`:

R code 7.25

```
set.seed(11)
WAIC( m6.7 )
```

```
      WAIC      lppd penalty  std_err
1 361.4511 -177.1724  3.5532 14.17035
```

The first value is the guess for the out-of-sample deviance. The other values are (in order): lppd, the effective number of parameters `penalty`, and the standard error of the WAIC value. The Overthinking box in the previous section shows how to calculate these numbers from

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

scratch. To make it easier to compare multiple models, the `rethinking` package provides a convenience function, `compare`:

R code 7.26

```
set.seed(77)
compare( m6.6 , m6.7 , m6.8 , func=WAIC )
```

```
      WAIC    SE dWAIC   dSE pWAIC weight
m6.7 361.9 14.26   0.0    NA   3.8      1
m6.8 402.8 11.28  40.9 10.48   2.6      0
m6.6 405.9 11.66  44.0 12.23   1.6      0
```

PSIS will give you almost identical values. You can add `func=PSIS` to the `compare` call to check. What do all of these numbers mean? Each row is a model. Columns from left to right are: WAIC, standard error (`SE`) of WAIC, difference of each WAIC from the best model, standard error (`dSE`) of this difference, prediction penalty (`pWAIC`), and finally the Akaike weight. Each of these needs a lot more explanation.

The first column contains the WAIC values. Smaller values are better, and the models are ordered by WAIC, from best to worst. The model that includes the fungus variable has the smallest WAIC, as promised. The `pWAIC` column is the penalty term of WAIC. These values are close to, but slightly below, the number of dimensions in the posterior of each model, which is to be expected in linear regressions with regularizing priors. These penalties are more interesting later on in the book.

The `dWAIC` column is the difference between each model's WAIC and the best WAIC in the set. So it's zero for the best model and then the differences with the other models tell you how far apart each is from the top model. So `m6.7` is about 40 units of deviance smaller than both other models. The intercept model, `m6.6`, is 3 units worse than `m6.8`. Are these big differences or small differences? One way to answer that is to ask a clearer question: Are the models easily distinguished by their expected out-of-sample accuracy? To answer that question, we need to consider the error in the WAIC estimates. Since we don't have the target sample, these are just guesses, and we know from the simulations that there is a lot of variation in WAIC's error.

That is what the two standard error columns, `SE` and `dSE`, are there to help us with. `SE` is the approximate standard error of each WAIC. In a very approximate sense, we expect the uncertainty in out-of-sample accuracy to be normally distributed with mean equal to the reported WAIC value and a standard deviation equal to the standard error. When the sample is small, this approximation tends to dramatically underestimate the uncertainty. But it is still better than older criteria like AIC, which provide no way to gauge their uncertainty.

Now to judge whether two models are easy to distinguish, we don't use their standard errors but rather the standard error of their difference. What does that mean? Just like each WAIC value, each difference in WAIC values also has a standard error. To compute the standard error of the difference between models `m6.7` and `m6.8`, we just need the pointwise breakdown of the WAIC values:

R code 7.27

```
set.seed(91)
waic_m6.7 <- WAIC( m6.7 , pointwise=TRUE )$WAIC
waic_m6.8 <- WAIC( m6.8 , pointwise=TRUE )$WAIC
n <- length(waic_m6.7)
diff_m6.7_m6.8 <- waic_m6.7 - waic_m6.8
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

```
sqrt( n*var( diff_m6.7_m6.8 ) )
```

```
[1] 10.35785
```

This is the value in the second row of the `compare` table. It's slightly different, only because of simulation variance. The difference between the models is 40.9 and the standard error is about 10.4. If we imagine the 99% (corresponding to a z-score of about 2.6) interval of the difference, it'll be about:

R code 7.28

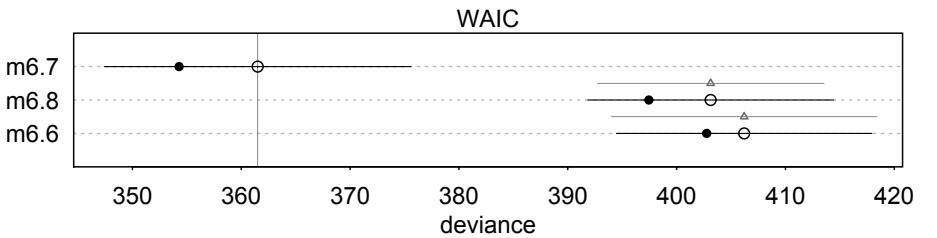
```
40.0 + c(-1,1)*10.4*2.6
```

```
[1] 12.96 67.04
```

So yes, these models are very easy to distinguish by expected out-of-sample accuracy. Model `m6.7` is a lot better. You might be able to see all of this better, if we plot the `compare` table:

R code 7.29

```
plot( compare( m6.6 , m6.7 , m6.8 ) )
```



The filled points are the in-sample deviance values. The open points are the WAIC values. Notice that naturally each model does better in-sample than it is expected to do out-of-sample. The line segments show the standard error of each WAIC. These are the values in the column labeled `SE` in the table above. So you can probably see how much better `m6.7` is than `m6.8`. What we really want however is the standard error of the difference in WAIC between the two models. That is shown by the lighter line segment with the triangle on it, between `m6.7` and `m6.8`.

What does all of this mean? It means that WAIC cannot be used to infer causation. We know, because we simulated these data, that the treatment matters. But because fungus mediates treatment—it is on a pipe between treatment and the outcome—once we condition on fungus, treatment provides no additional information. And since fungus is more highly correlated with the outcome, a model using it is likely to predict better. WAIC did its job. Its job is not to infer causation. Its job is to guess predictive accuracy.

That doesn't mean that WAIC (or CV or PSIS) is useless here. It does provide a useful measure of the expected improvement in prediction that comes from conditioning on the fungus. Although the treatment works, it isn't 100% effective, and so knowing the treatment is no substitute for knowing whether fungus is present.

Similarly, we can ask about the difference between models `m6.8`, the model with treatment only, and model `m6.6`, the intercept model. Model `m6.8` provides pretty good evidence that the treatment works. You can inspect the posterior again, if you have forgotten. But WAIC thinks these two models are quite similar. Their difference is only 3 units of deviance. Let's calculate the standard error of the difference, to highlight the issue:

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

R code 7.30

```
set.seed(92)
waic_m6.6 <- WAIC( m6.6 , pointwise=TRUE )$WAIC
diff_m6.6_m6.8 <- waic_m6.6 - waic_m6.8
sqrt( n*var( diff_m6.6_m6.8 ) )
```

```
[1] 4.858914
```

The `compare` table doesn't show this value, but it did calculate it. To see it, you need the `dSE` slot of the return:

R code 7.31

```
set.seed(93)
compare( m6.6 , m6.7 , m6.8 )@dSE
```

```
          m6.6     m6.7      m6.8
m6.6        NA 12.20638  4.934353
m6.7 12.206380       NA 10.426576
m6.8  4.934353 10.42658        NA
```

This matrix contains all of the pairwise difference standard errors for the models you compared. Notice that the standard error of the difference for `m6.6` and `m6.8` is bigger than the difference itself. We really cannot easily distinguish these models on the basis of WAIC. Note that these contrasts are possibly less reliable than the standard errors on each model. There isn't much analytical work on these contrasts yet, but before long there should be.[129]

Does this mean that the treatment doesn't work? Of course not. We know that it works. We simulated the data. And the posterior distribution of the treatment effect, `bt` in `m6.8`, is reliably positive. But it isn't especially large. So it doesn't do much alone to improve prediction of plant height. There are just too many other sources of variation. This result just echoes the core fact about WAIC (and CV and PSIS): It guesses predictive accuracy, not causal truth. A variable can be causally related to an outcome, but have little relative impact on it, and WAIC will tell you that. That is what is happening in this case. We can use WAIC/CV/PSIS to measure how big a difference some variable makes in prediction. But we cannot use these criteria to decide whether or not some effect exists. We need the posterior distributions of multiple models, maybe examining the implied conditional independencies of a relevant causal graph, to do that.

The last element of the `compare` table is the column we skipped over, `weight`. These values are a traditional way to summarize relative support for each model. They always sum to 1, within a set of compared models. The weight of a model i is computed as:

$$w_i = \frac{\exp(-0.5\Delta_i)}{\sum_j \exp(-0.5\Delta_j)}$$

where Δ_i is the difference between model i's WAIC value and the best WAIC in the set. These are the `dWAIC` values in the table. These weights can be a quick way to see how big the differences are among models. But you still have to inspect the standard errors. Since the weights don't reflect the standard errors, they are simply not sufficient for model comparison. Weights are also used in **MODEL AVERAGING**. Model averaging is a family of methods for combining the predictions of multiple models. For the sake of space, we won't cover it in this book. But see the endnote for some places to start.[130]

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

Rethinking: WAIC metaphors. Here are two metaphors to help explain the concepts behind using WAIC (or another information criterion) to compare models.

Think of models as race horses. In any particular race, the best horse may not win. But it's more likely to win than is the worst horse. And when the winning horse finishes in half the time of the second-place horse, you can be pretty sure the winning horse is also the best. But if instead it's a photo-finish, with a near tie between first and second place, then it is much harder to be confident about which is the best horse. WAIC values are analogous to these race times—smaller values are better, and the distances between the horses/models are informative. Akaike weights transform differences in finishing time into probabilities of being the best model/horse on future data/races. But if the track conditions or jockey changes, these probabilities may mislead. Forecasting future racing/prediction based upon a single race/fit carries no guarantees.

Think of models as stones thrown to skip on a pond. No stone will ever reach the other side (perfect prediction), but some sorts of stones make it farther than others, on average (make better test predictions). But on any individual throw, lots of unique conditions avail—the wind might pick up or change direction, a duck could surface to intercept the stone, or the thrower's grip might slip. So which stone will go farthest is not certain. Still, the relative distances reached by each stone therefore provide information about which stone will do best on average. But we can't be too confident about any individual stone, unless the distances between stones is very large.

Of course neither metaphor is perfect. Metaphors never are. But many people find these to be helpful in interpreting information criteria.

7.5.2. Outliers and other illusions. In the divorce example from Chapter 5, we saw in the posterior predictions that a few States were very hard for the model to retrodict. The State of Idaho in particular was something of an **OUTLIER** (page 5.5). Individual points like Idaho tend to be very influential in ordinary regression models. Let's see how PSIS and WAIC represent that importance. Begin by refitting the three divorce models from Chapter 5.

R code 7.32

```
library(rethinking)
data(WaffleDivorce)
d <- WaffleDivorce
d$A <- standardize( d$MedianAgeMarriage )
d$D <- standardize( d$Divorce )
d$M <- standardize( d$Marriage )

m5.1 <- quap(
    alist(
        D ~ dnorm( mu , sigma ) ,
        mu <- a + bA * A ,
        a ~ dnorm( 0 , 0.2 ) ,
        bA ~ dnorm( 0 , 0.5 ) ,
        sigma ~ dexp( 1 )
    ) , data = d )

m5.2 <- quap(
    alist(
        D ~ dnorm( mu , sigma ) ,
        mu <- a + bM * M ,
        a ~ dnorm( 0 , 0.2 ) ,
```

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

```
        bM ~ dnorm( 0 , 0.5 ) ,
        sigma ~ dexp( 1 )
    ) , data = d )

m5.3 <- quap(
    alist(
        D ~ dnorm( mu , sigma ) ,
        mu <- a + bM*M + bA*A ,
        a ~ dnorm( 0 , 0.2 ) ,
        bM ~ dnorm( 0 , 0.5 ) ,
        bA ~ dnorm( 0 , 0.5 ) ,
        sigma ~ dexp( 1 )
    ) , data = d )
```

Look at the posterior summaries, just to remind yourself that marriage rate (M) has little association with divorce rate (D), once age at marriage (A) is included in `m5.3`. Now let's compare these models using PSIS:

R code 7.33

```
set.seed(24071847)
compare( m5.1 , m5.2 , m5.3 , func=PSIS )
```

```
      PSIS    SE dPSIS   dSE pPSIS weight
m5.1 127.6 14.69   0.0    NA   4.7   0.71
m5.3 129.4 15.10   1.8  0.90   5.9   0.29
m5.2 140.6 11.21  13.1 10.82   3.8   0.00
```

There are two important things to consider here. First note that the model that omits marriage rate, `m5.1`, lands on top. This is because marriage rate has very little association with the outcome. So the model that omits it has slightly better expected out-of-sample performance, even though it actually fits the sample slightly worse than `m5.3`, the model with both predictors. The difference between the top two models is only 1.8, with a standard error of 0.9, so the models make very similar predictions. This is the typical pattern, whenever some predictor has a very small association with the outcome.

Second, in addition to the table above, you should also receive a message:

```
Some Pareto k values are very high (>1).
```

This means that the smoothing approximation that PSIS uses is unreliable for some points. Recall from the section on PSIS that when a point's Pareto k value is above 0.5, the importance weight can be unreliable. Furthermore, these points tend to be outliers with unlikely values, according to the model. As a result, they are highly influential and make it difficult to estimate out-of-sample predictive accuracy. Why? Because any new sample is unlikely to contain these same outliers, and since these outliers were highly influential, they could make out-of-sample predictions worse than expected. WAIC is vulnerable to outliers as well. It doesn't have an automatic warning. But it does have a way to measure this risk, through the estimate of the overfitting penalty.

Let's look at the individual States, to see which are causing the problem. We can do this by adding `pointwise=TRUE` to `PSIS`. When you do this, you get a matrix with each observation on a row and the PSIS information, including individual Pareto k values, in columns. I'll also

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

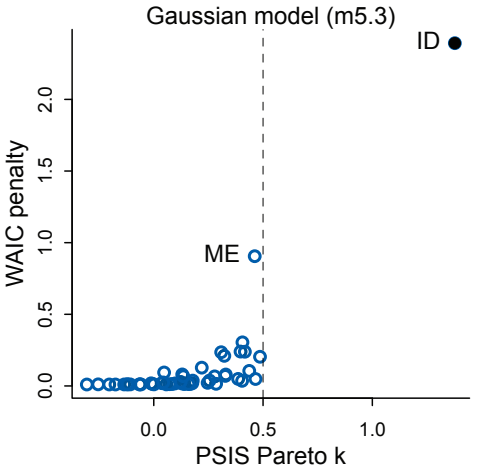


FIGURE 7.10. Highly influential points and out-of-sample prediction. The horizontal axis is Pareto *k* from PSIS. The vertical axis is WAIC's penalty term. The State of Idaho (ID) has an extremely unlikely value, according to the model. As a result it has both a very high Pareto *k* and a large WAIC penalty. Points like these are highly influential and potentially hurt prediction.

plot the individual "penalty" values from WAIC, to show the relationship between Pareto *k* and the information theoretic prediction penalty.

R code 7.34

```
set.seed(24071847)
PSIS_m5.3 <- PSIS(m5.3,pointwise=TRUE)
set.seed(24071847)
WAIC_m5.3 <- WAIC(m5.3,pointwise=TRUE)
plot( PSIS_m5.3$k , WAIC_m5.3$penalty , xlab="PSIS Pareto k" ,
    ylab="WAIC penalty" , col=rangi2 , lwd=2 )
```

This plot is shown in FIGURE 7.10. Individual points are individual States, with Pareto *k* on the horizontal axis and WAIC's penalty term. The State of Idaho (ID, upper-right corner) has both a very high Pareto *k* value (above 1) and a large penalty term (over 2). As you saw back in Chapter 5, Idaho has a very low divorce rate for its age at marriage. As a result, it is highly influential—it exerts more influence on the posterior distribution than other States do. The Pareto *k* value is double the theoretical point at which the variance becomes infinite (shown by the dashed line). Likewise, WAIC assigns Idaho a penalty over 2. This penalty term is sometimes called the "effective number of parameters," because in ordinary linear regressions the sum of all penalty terms from all points tends to be equal to the number of free parameters in the model. But in this case there are 4 parameters and the total penalty is closer to 6—check `WAIC(m5.3)`. The outlier Idaho is causing this additional overfitting risk.

What can be done about this? There is a tradition of dropping outliers. People sometimes drop outliers even before a model is fit, based only on standard deviations from the mean outcome value. You should never do that—a point can only be unexpected and highly influential in light of a model. After you fit a model, the picture changes. If there are only a few outliers, and you are sure to report results both with and without them, dropping outliers might be okay. But if there are several outliers and we really need to model them, what then?

A basic problem here is that the Gaussian error model is easily surprised. Gaussian distributions (introduced at the start of Chapter 4) have very thin tails. This means that very little probability mass is given to observations far from the mean. Many natural phenomena do have very thin tails like this. Human height is a good example. But many phenomena do

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

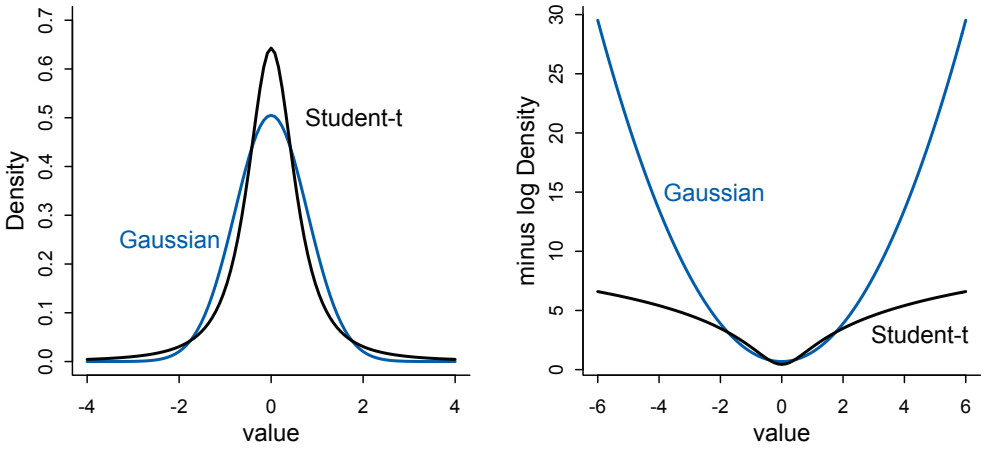


FIGURE 7.11. Thin tails and influential observations. The Gaussian distribution (blue) assigns very little probability to extreme observations. It has thin tails. The Student-t distribution with shape $\nu = 2$ (black) assigns more probability to extreme events. These distributions are compared on the probability (left) and log-probability (right) scales.

not. Instead many phenomena have thicker tails with rare, extreme observations. These are not measurement errors, but real events containing information about natural process.

One way to both use these extreme observations and reduce their influence is to employ some kind of **ROBUST REGRESSION**. A "robust regression" can mean many different things, but usually it indicates a linear model in which the influence of extreme observations is reduced. A common and useful kind of robust regression is to replace the Gaussian model with a thicker-tailed distribution like **STUDENT'S T** (or "Student-t") distribution.[131] This distribution has nothing to do with students. The Student-t distribution arises from a mixture of Gaussian distributions with different variances.[132] If the variances are diverse, then the tails can be quite thick.

The generalized Student-t distribution has the same mean μ and scale σ parameters as the Gaussian, but it also has an extra shape parameter ν that controls how thick the tails are. The `rethinking` package provides Student-t as `dstudent`. When ν is large, the tails are thin, converging in the limit $\nu = \infty$ to a Gaussian distribution. But as ν approaches 1, the tails get thicker and rare extreme observations occur more often. FIGURE 7.11 compares a Gaussian distribution (in blue) to a corresponding Student-t distribution (in black) with $\nu = 2$. The Student-t distribution has thicker tails, and this is most obvious on the log scale (right), where the Gaussian tails shrink quadratically—a normal distribution is just an exponentiated parabola remember—while the Student-t tails shrink much more slowly.

If you have a very large data set with such events, you could estimate ν. Financial time series, taken over very long periods, are one example. But when using robust regression, we don't usually try to estimate ν, because there aren't enough extreme observations to do so. Instead we assume ν is small (thick tails) in order to reduce the influence of outliers. For example, if we use the severity of wars since 1950 to estimate a trend, the estimate is likely biased by the fact that big conflicts like the first and second World Wars are rare. They reside

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

in the thick tail of war casualties.[133] A reasonable estimate depends upon either a longer time series or judicious use of a thick tailed distribution.

Let's re-estimate the divorce model using a Student-t distribution with $\nu = 2$.

R code 7.35

```
m5.3t <- quap(
    alist(
        D ~ dstudent( 2 , mu , sigma ) ,
        mu <- a + bM*M + bA*A ,
        a ~ dnorm( 0 , 0.2 ) ,
        bM ~ dnorm( 0 , 0.5 ) ,
        bA ~ dnorm( 0 , 0.5 ) ,
        sigma ~ dexp( 1 )
    ) , data = d )
```

When you compute PSIS now, `PSIS(m5.3t)`, you won't get any warnings about Pareto k values. The relative influence of Idaho has been much reduced. How does this impact the posterior distribution of the association between age at marriage and divorce? If you compare models `m5.3t` and `m5.3`, you'll see that the coefficient `bA` has gotten farther from zero when we introduce the Student-t distribution. This is because Idaho has a low divorce rate and a low median age at marriage. When it was influential, it reduced the association between age at marriage and divorce. Now it is less influential, so the association is estimated to be slightly larger. But the consequence of using robust regression is not always to increase an association. It depends upon the details.

Another thing that thick-tailed distributions make possible is control over how conflict between prior and data is handled. We'll revisit this point in a later chapter, once you have started using Markov chains and can derive non-Gaussian posterior distributions.

Rethinking: The Curse of Tippecanoe. One concern with model comparison is, if we try enough combinations and transformations of predictors, we might eventually find a model that fits any sample very well. But this fit will be badly overfit, unlikely to generalize. And WAIC and similar metrics will be fooled. Consider by analogy the *Curse of Tippecanoe*.[134] From the year 1840 until 1960, every United States president who was elected in a year ending in the digit 0 (which happens every 20 years) has died in office. William Henry Harrison was the first, elected in 1840 and died of pneumonia the next year. John F. Kennedy was the last, elected in 1960 and assassinated in 1963. Seven American presidents died in sequence in this pattern. Ronald Reagan was elected in 1980, but despite at least one attempt on his life, he managed to live long after his term, breaking the curse. Given enough time and data, a pattern like this can be found for almost any body of data. If we search hard enough, we are bound to find a Curse of Tippecanoe.

Fiddling with and constructing many predictor variables is a great way to find coincidences, but not necessarily a great way to evaluate hypotheses. However, fitting many possible models isn't always a dangerous idea, provided some judgment is exercised in weeding down the list of variables at the start. There are two scenarios in which this strategy appears defensible. First, sometimes all one wants to do is explore a set of data, because there are no clear hypotheses to evaluate. This is rightly labeled pejoratively as **DATA DREDGING**, when one does not admit to it. But when used together with model averaging, and freely admitted, it can be a way to stimulate future investigation. Second, sometimes we need to convince an audience that we have tried all of the combinations of predictors, because none of the variables seem to help much in prediction.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

7.6. Summary

This chapter has been a marathon. It began with the problem of overfitting, a universal phenomenon by which models with more parameters fit a sample better, even when the additional parameters are meaningless. Two common tools were introduced to address overfitting: regularizing priors and estimates of out-of-sample accuracy (WAIC and PSIS). Regularizing priors reduce overfitting during estimation, and WAIC and PSIS help estimate the degree of overfitting. Practical functions `compare` in the `rethinking` package were introduced to help analyze collections of models fit to the same data. If you are after causal estimates, then these tools will mislead you. So models must be designed through some other method, not selected on the basis of out-of-sample predictive accuracy. But any causal estimate will still overfit the sample. So you always have to worry about overfitting, measuring it with WAIC/PSIS and reducing it with regularization.

7.7. Practice

Problems are labeled Easy (**E**), Medium (**M**), and Hard (**H**).

7E1. State the three motivating criteria that define information entropy. Try to express each in your own words.

7E2. Suppose a coin is weighted such that, when it is tossed and lands on a table, it comes up heads 70% of the time. What is the entropy of this coin?

7E3. Suppose a four-sided die is loaded such that, when tossed onto a table, it shows "1" 20%, "2" 25%, "3" 25%, and "4" 30% of the time. What is the entropy of this die?

7E4. Suppose another four-sided die is loaded such that it never shows "4". The other three sides show equally often. What is the entropy of this die?

7M1. Write down and compare the definitions of AIC and WAIC. Which of these criteria is most general? Which assumptions are required to transform the more general criterion into a less general one?

7M2. Explain the difference between model *selection* and model *comparison*. What information is lost under model selection?

7M3. When comparing models with an information criterion, why must all models be fit to exactly the same observations? What would happen to the information criterion values, if the models were fit to different numbers of observations? Perform some experiments, if you are not sure.

7M4. What happens to the effective number of parameters, as measured by PSIS or WAIC, as a prior becomes more concentrated? Why? Perform some experiments, if you are not sure.

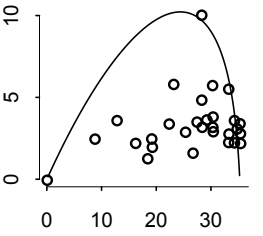
7M5. Provide an informal explanation of why informative priors reduce overfitting.

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.

7M6. Provide an informal explanation of why overly informative priors result in underfitting.

7H1. In 2007, *The Wall Street Journal* published an editorial ("We're Number One, Alas") with a graph of corporate tax rates in 29 countries plotted against tax revenue. A badly fit curve was drawn in (reconstructed at right), seemingly by hand, to make the argument that the relationship between tax rate and tax revenue increases and then declines, such that higher tax rates can actually produce less tax revenue. I want you to actually fit a curve to these data, found in `data(Laffer)`. Consider models that use tax rate to predict tax revenue. Compare, using WAIC or PSIS, a straight-line model to any curved models you like. What do you conclude about the relationship between tax rate and tax revenue?



7H2. In the `Laffer` data, there is one country with a high tax revenue that is an outlier. Use PSIS and WAIC to measure the importance of this outlier in the models you fit in the previous problem. Then use robust regression with a Student's t distribution to revisit the curve fitting problem. How much does a curved relationship depend upon the outlier point?

7H3. Consider three fictional Polynesian islands. On each there is a Royal Ornithologist charged by the king with surveying the bird population. They have each found the following proportions of 5 important bird species:

	Species A	Species B	Species C	Species D	Species E
Island 1	0.2	0.2	0.2	0.2	0.2
Island 2	0.8	0.1	0.05	0.025	0.025
Island 3	0.05	0.15	0.7	0.05	0.05

Notice that each row sums to 1, all the birds. This problem has two parts. It is not computationally complicated. But it is conceptually tricky. First, compute the entropy of each island's bird distribution. Interpret these entropy values. Second, use each island's bird distribution to predict the other two. This means to compute the KL divergence of each island from the others, treating each island as if it were a statistical model of the other islands. You should end up with 6 different KL divergence values. Which island predicts the others best? Why?

7H4. Recall the marriage, age, and happiness collider bias example from Chapter 6. Run models `m6.9` and `m6.10` again (page 178). Compare these two models using WAIC (or PSIS, they will produce identical results). Which model is expected to make better predictions? Which model provides the correct causal inference about the influence of age on happiness? Can you explain why the answers to these two questions disagree?

7H5. Revisit the urban fox data, `data(foxes)`, from the previous chapter's practice problems. Use WAIC or PSIS based model comparison on five different models, each using `weight` as the outcome, and containing these sets of predictor variables:

(1) `avgfood + groupsize + area`
(2) `avgfood + groupsize`
(3) `groupsize + area`
(4) `avgfood`
(5) `area`

Can you explain the relative differences in WAIC scores, using the fox DAG from the previous chapter? Be sure to pay attention to the standard error of the score differences (`dSE`).

Copyright © 2020. CRC Press LLC. All rights reserved.

McElreath, Richard. Statistical Rethinking : A Bayesian Course with Examples in R and STAN, CRC Press LLC, 2020. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/massey/detail.action?docID=6133700.
Created from massey on 2022-07-11 16:25:15.